

Exact Solution of the Vehicle Routing Problem with Drones

Jeanette Schmidt,^{a,*} Christian Tilk,^b Stefan Irnich^a

^aChair of Logistics Management, Department of Business and Economics, Johannes Gutenberg University Mainz, D-55128 Mainz, Germany;

^bDepartment of Business Decisions and Analytics, University of Vienna, AT-1090 Vienna, Austria

*Corresponding author

Contact: sjeanett@uni-mainz.de,  <https://orcid.org/0000-0001-7565-5093> (JS); christian.tilk@univie.ac.at,

 <https://orcid.org/0000-0003-4906-3160> (CT); irnich@uni-mainz.de,  <https://orcid.org/0000-0001-9383-4546> (SI)

Received: May 23, 2023

Revised: January 24, 2024; July 22, 2024;
October 10, 2024

Accepted: October 19, 2024

Published Online in Articles in Advance:
November 28, 2024

<https://doi.org/10.1287/trsc.2024.0544>

Copyright: © 2024 INFORMS

Abstract. The vehicle routing problem with drones (VRP-D) that we consider is an extension of the capacitated vehicle routing problem in which the fleet consists of trucks equipped with one drone each. A truck and its drone can either move together or separately. A truck can release its drone at the depot or at a customer location and must pick it up later at another customer or the depot location. In this way, both trucks and drones deliver goods to customers working together as synchronized working units. A feasible route has to satisfy the capacity constraints of both the truck and the drone. A feasible solution to the VRP-D is a set of feasible routes such that each customer is served exactly once by either a truck or a drone. We investigate two standard objectives considered in the literature, that is, the minimization of the total routing cost and the sum of the routes' durations. To solve the VRP-D exactly, we develop a branch-price-and-cut (BPC) algorithm. In particular, we present a new forward and implicit bidirectional labeling algorithm defined over an artificial network to solve the column-generation subproblems. The new bidirectional labeling algorithm substantially accelerates the solution process compared with its monodirectional counterpart. The time needed to solve the pricing problems is reduced by 55% on average when minimizing routing costs and by 30% when minimizing the sum of the routes' durations. In further computational experiments, we analyze algorithmic components of the BPC algorithm, compare the cost and duration objectives, and highlight the impact of the drones' speed on the structure of VRP-D solutions. For the routing-cost minimization objective, our BPC algorithm is able to solve several VRP-D instances with 50 vertices to proven optimality within one hour of computation time. The same instances with duration minimization are more difficult, and the BPC algorithm provides only heuristic solutions with an average gap not exceeding 3%.

Funding: This work was supported by Deutsche Forschungsgemeinschaft [Project 418727865, Grant IR 122/10-1].

Supplemental Material: The online appendix is available at <https://doi.org/10.1287/trsc.2024.0544>.

Keywords: routing • drone delivery • synchronization • branch-price-and-cut • bidirectional labeling

1. Introduction

Drones are an emerging technology. This explains why planning problems related to drones have attracted researchers from various disciplines, for example, engineering, computer science, and control theory (Otto et al. 2018). A *drone*, a.k.a. unmanned aerial vehicle, is an aircraft that does not have a pilot onboard and is controlled remotely (International Trade Administration 2023). Compared with ground and water vehicles such as trucks and ships, modern drones are typically faster, can move in three dimensions to surmount obstacles on the ground, can take off and land autonomously, have a lower cost per kilometer to operate, and emit less CO₂ (D'Andrea 2014, Goodchild and Toy 2018). In this sense, the use of drones represents a greener and more versatile alternative to conventional delivery modes. However, they have the drawback of a limited payload and

flight range (even if modern parcel delivery drones can fly up to several kilometers with a payload of about five kilograms, more than 5% of parcels weigh between 15 and 30 kilograms (Joerss et al. 2016)). We consider combined *delivery operations* of trucks and drones, which can lead to a significant increase of efficiency and effectiveness in performing the delivery operations (Wang, Pökönen, and Golden 2017, Carlsson and Song 2018). According to Otto et al. (2018, section 5), such combined truck-and-drone operations are of four types: (1) trucks support the delivery operations performed solely by drones, (2) drones support the delivery operations performed solely by trucks, (3) trucks and drones operate independently and perform separate delivery routes, and (4) trucks and drones work together as synchronized working units. Combined delivery operations make particular sense when conventional vehicles

cannot easily and quickly reach some intermediate or delivery points, for example, for parcel delivery to remote households, medical supply delivery (vaccines, blood, drugs) and collection (samples of blood, urine, etc.), and delivery of spare parts. For the numerous existing and potential applications, we refer the reader to the surveys (Otto et al. 2018, Chung, Sah, and Lee 2020, Macrina et al. 2020, Moshref-Javadi and Winkenbach 2021, Madani and Ndiaye 2022). We focus on the synchronized routing problem of type (4).

The variant of the *vehicle routing problem with drones* (VRP-D) that we consider has a fleet of trucks that is equipped with one drone each. A truck and its drone can either move together or separately. When truck and drone move together, the drone is either inside or on top of the truck. The drone can leave the truck at the depot or any customer location and perform a single delivery alone (this is realistic given the limited payload of the drone and for operational/safety reasons). Meanwhile, the truck continues its journey alone, serving one or several other customers. In any case, the drone must return to the same truck at a customer or the depot location. The VRP-D consists of finding feasible combined truck-and-drone routes to serve a given set of customers. A feasible route must satisfy the following conditions: (1) it starts and ends at the given depot, (2) the capacity of the truck is respected, and (3) each drone flight serves exactly one customer. Common objectives of the VRP-D are minimizing the total routing cost and the sum of the durations of all routes. The literature does not consider and discuss that these two objectives are competing. Hence, we will analyze the different structures of solutions obtained with each objective (see Section 5.4).

The need to synchronize the movements of trucks and drones makes the VRP-D more challenging compared with the classical *capacitated vehicle routing problem* (CVRP) (Irnich, Toth, and Vigo 2014) because additional decisions about which type of vehicle serves which subset of customers must be made (called *task synchronization* in Drexel 2012). The synchronization of a truck and its drone in space (*movement synchronization*) and, for duration minimization, synchronization in time (*operation synchronization*) must be considered.

We develop a VRP-D-tailored *branch-price-and-cut* (BPC) algorithm that allows the exact solution for both objectives, that is, the minimization of the total routing cost and the sum of the durations. Note that in a multiple-vehicle context without time windows, the minimization of duration refers to the sum of the completion times of all routes where each route starts at time 0. It should not be confused with the minimization of the makespan, which is the minimization of the latest completion time and leads to a problem of min-max type.

The most important algorithmic component of the BPC algorithm is a bidirectional labeling algorithm to

solve the column-generation subproblems. This subproblem is defined over an artificial network originally introduced by Roberti and Ruthmair (2021) for the single-vehicle version of this problem, that is, for the *traveling salesman problem with drone* (TSP-D). Whereas Roberti and Ruthmair use a nontrivial monodirectional labeling algorithm, we use a bidirectional labeling algorithm that is responsible for the convincing performance of the overall BPC algorithm; we are able to tackle larger instances than solved for the TSP-D, which is opposed to the often observed relationship between the practical difficulty of single- and multiple-vehicle routing problems.

The artificial network is a multidigraph in which vertices represent possible combinations of locations where a truck and its associated drone perform their service. Arcs of the artificial network model possible truck-and-drone movements. A major complication in designing an effective bidirectional labeling algorithm is that the same route, once considered in the forward and once in the backward direction, passes through different vertices of the artificial network whenever the drone operates without the truck. In particular, the merge procedure is unclear in such a setting. Moreover, the subproblem is asymmetric even though the VRP-D is defined as a symmetric routing problem. Unexpectedly, the inherent symmetry of the VRP-D can be exploited: instead of explicitly generating forward and backward labels, the same set of labels can be considered for both directions. This is what we name *implicit* bidirectional labeling.

Accordingly, the main contributions of the paper at hand are as follows:

- We develop a new exact algorithm based on BPC to solve the VRP-D exactly for both objectives, minimization of routing cost and minimization of the sum of the routes' durations.
- We show that an implicit bidirectional labeling algorithm is essential for solving the pricing subproblems effectively. A particular novelty is that we apply two different merge procedures in the bidirectional labeling algorithm depending on whether the truck and its drone are traveling together (or separately) at the merge point.
- We present several computational analyses. First, we evaluate the new algorithmic components suggested to accelerate the merge procedure that constitutes the bottleneck operation of the overall BPC algorithm. Second, monodirectional labeling is compared with the implicit bidirectional labeling. Moreover, we compare the two objectives and the impact of the drone and truck's speeds and routing costs on the structure and quality of solutions. Finally, results and analyses with TSP-D and VRP-D instances from the literature and with the VRP-D variant with limited flight range are presented. They show that the BPC algorithm is widely applicable.

The remainder of this work is structured as follows. We review the pertinent literature in Section 2. Section 3 formally introduces the VRP-D. In Section 4, we present the new BPC algorithm to solve the VRP-D, with a particular focus on the implicit bidirectional labeling algorithm. Computational results are discussed in Section 5, and final conclusions are drawn in Section 6.

2. Literature Review

The integration of drones in vehicle routing is receiving increasing attention in the research literature (Otto et al. 2018, Chung, Sah, and Lee 2020, Macrina et al. 2020, Moshref-Javadi and Winkenbach 2021, Madani and Ndiaye 2022). These surveys show the vast amount of scientific contributions that have been published in recent years, where heuristic solution approaches are predominant. For the sake of brevity, we limit the scope of the literature review to exact optimization approaches in the context of TSP-D and VRP-D variants of type (4) in the classification of Otto et al. (2018), that is, when trucks and drones operate together as synchronized working units.

Roberti and Ruthmair (2021) focus on the basic variant of the TSP-D, that is, a single truck hosting a single drone that can carry not more than one package and hence serve only one drone customer per launch from the truck. The objective is the minimization of the route's duration. The methodological contributions of Roberti and Ruthmair (2021) are the presentation and computational analysis of a *mixed-integer programming* (MIP) formulation and a *dynamic-programming* (DP) formulation for both the basic TSP-D and several extensions. Instances with up to 39 customers are solved with a branch-and-price algorithm that uses an *ng*-route relaxation of the DP formulation. These results can be considered a milestone, as prior to that, only much smaller instances could be solved exactly. Moreover, their work provides an excellent overview of TSP-D variants so that we can be brief with respect to the single-truck problems. Only the very recent approach for the TSP-D by Blufstein, Lera-Romero, and Soullignac (2024), which was developed in parallel with our work, must be mentioned: the authors develop an exact algorithm based on a decremental state-space relaxation. As we do, they derive a bidirectional labeling algorithm that is based on the network of Roberti and Ruthmair (2021). However, their price-fix-and-augment method is tailored to the TSP-D and applies a sophisticated combination of column generation, variable fixing using completion bounds, and a *ng*-neighborhood extension to solve the TSP-D. They introduce a restricted notion of *ng*-feasibility that leads to improved lower bounds (compared with Roberti and Ruthmair 2021) that result from the solution of the associated dynamic-programming relaxations. Additionally, a partial dominance rule

make use of the new notion and further accelerates the solution of the dynamic programs. It is remarkable that their algorithm solves every TSP-D benchmark instance of the testbed with up to 59 customers. Unfortunately, their algorithm is not directly applicable to the VRP-D. Obviously, their underlying state space has different states for different tour lengths and does not need to keep track of the accumulated load. More important is that their variable fixing fully exploits the TSP property that *the* route visits each customer exactly once. Moreover, the exploitation of tentative upper bounds of the TSP-D (for variable fixing) cannot be used when solving multiple pricing problems as needed in BPC.

The first article dealing with a VRP-D variant of type (4) was published by Wang, Poikonen, and Golden (2017). They define the VRP-D as a generalization of the TSP-D by using multiple trucks each equipped with multiple drones. Wang, Poikonen, and Golden (2017) assume that each drone is assigned to a specific truck and that a drone can only be released and picked up at either a customer location or the depot. The objective is to minimize the sum of the durations of the routes. Besides the introduction of the VRP-D, they conduct comprehensive analyses of worst-case scenarios on the benefits of using trucks and drones as synchronized work units instead of truck-only solutions. The same authors later extended their analyses in a follow-up paper (Poikonen, Wang, and Golden 2017). Bakir and Tiniç (2020) consider the VRP-D with flexible drones; that is, a drone does not necessarily have to return to the truck from where it was released. They formulate the problem as a *mixed-integer linear program* (MILP) on a time-space network and propose a dynamic discretization discovery approach that solves instances with up to 25 customers within five hours. Tamke and Buscher (2021) use a branch-and-cut algorithm to solve a version of the VRP-D in which each truck can be equipped with more than one drone. For larger instances, the number of drones on each truck is limited to two drones. Instances with up to 29 customers are solved within 12 hours. Zhen et al. (2023) developed a BPC algorithm to solve the VRP-D with the objective to minimize the total routing cost. They can solve instances with up to 14 customers to proven optimality within a three-hour time limit. In the publication by Zhou et al. (2023), each truck is equipped with more than one drone that is uniquely assigned to this truck. The truck has to wait at a customer location until all launched drones return before serving the next customer. Such a direct return policy also simplifies the synchronization between trucks and drones in the branch-and-price algorithms of Zhou et al. (2023). For this setting with a simple synchronization, the authors develop a branch-and-price algorithm that uses a bidirectional labeling algorithm that solves some instances with up to 35 customers within three hours. Li and Wang (2022) extend the VRP-D by

considering additional customer time windows. To the best of our knowledge, they propose the first exact solution approach, which is a BPC algorithm for this VRP-D variant. Instances with up to 50 customers are solved to optimality within 20 hours. To solve larger instances, they combine the BPC algorithm with an adaptive large-neighborhood search. In the BPC approaches of Wang and Sheu (2019) and Xia et al. (2023), the practically and methodologically difficult rigid synchronization between trucks and drones is relaxed with the introduction of *drone hubs* (or *docking hub*); that is, the drones do not return to the trucks.

The heuristic community has introduced many VRP-D variants. The existing exact approaches cover only some of these variants, but they all consume considerable computation time (several hours) to solve small-sized instances to optimality. From an exact solution perspective, the planning of combined delivery operations with trucks and drones collaborating in a synchronized way is very challenging (Drexel 2012). We present a faster exact BPC solution algorithm for the basic version of the VRP-D in which trucks and drones work together as synchronized working units. We consider duration and routing cost objectives and show that many variants discussed in the literature can be solved similarly.

3. The Vehicle Routing Problem with Drones

We define the VRP-D on an undirected complete graph (V, E) with vertex set V and edge set E . The vertex set V comprises $\{0, 0'\} \cup N$, where 0 and $0'$ represent the *depot* (two copies for start point and endpoint), and $N = \{1, 2, \dots, m\}$ denotes the set of *customers*. Each customer $v \in N$ has a positive integer *demand* q_v , which has to be served by either a truck or a drone. If the demand q_v exceeds a given threshold $\bar{q}$, v is called a *truck-only customer*, which cannot be served by drone.

A fleet of K homogeneous trucks is stationed at the depot. Each truck has a *capacity* Q and is equipped with a single drone (the assignment of the drone to its truck is assumed fixed). The truck and its drone can either move together or separately. When truck and drone move together, only the truck can *serve* customers. The drone can leave the truck at the depot 0 or at any customer location to operate alone. In the meantime, the truck continues its journey and can serve other customers on its own. The drone must return to the same truck at the truck's current location, which is either the location of a customer served by the truck or the depot $0'$. Note that the drone is not allowed to leave and return to the truck at the same customer. Moreover, each drone has a limited capacity in the sense that it can serve at most one customer before returning back to its truck. Accordingly, a *route* in the VRP-D is a pair $r = (P, D)$ consisting

of the *truck path* P and a possibly empty sequence D of *drone subpaths*. The truck path $P = (v_0, v_1, \dots, v_\ell, v_{\ell+1})$ starts at the depot $v_0 = 0$, serves the customers $v_p \in N$ for all $p \in \{1, \dots, \ell\}$ in this sequence, and ends at the depot $v_{\ell+1} = 0'$. The drone subpaths $D = (D^1, \dots, D^d)$ specify the drone flights along the truck path P . For each index $s \in \{1, \dots, d\}$, each subpath $D^s = \langle v^s, k^s, w^s \rangle$ contains exactly three vertices: at $v^s \in \{0\} \cup N$, the drone leaves the truck, $k^s \in N$ specifies the customer that is served by the drone, and $w^s \in N \cup \{0'\}$ indicates where the drone lands on the truck again. In a feasible route $r = (P, D)$, there exist, for each index $s \in \{1, 2, \dots, d\}$, two indices $g, h \in \{0, 1, \dots, \ell, \ell + 1\}$ with $g < h$ such that $v^s = v_g$ and $w^s = v_h$ holds. For convenience, we define $I(s) = \{g, g + 1, \dots, h - 1\}$ as the positions (indices) of P where either the drone leaves the truck or the truck serves a customer alone (intentionally, the position where the drone lands on the truck is not included). If a route contains more than one drone subpath, that is, $d > 1$, the sets $I(s)$ and $I(s')$ for $s \neq s'$, $1 \leq s, s' \leq d$ must have empty intersection (the segments where the truck is alone do not overlap). Finally, the route $r = (P, D)$ is *feasible* if $\sum_{p=1}^{\ell} q_{v_p} + \sum_{s=1}^d q_{k^s} \leq Q$ and $q_{k^s} \leq \bar{q}$ for all $s \in \{1, \dots, d\}$. The route $r = (P, D)$ is *elementary* if $v_1, \dots, v_\ell$ and $k^1, \dots, k^d$ are all different.

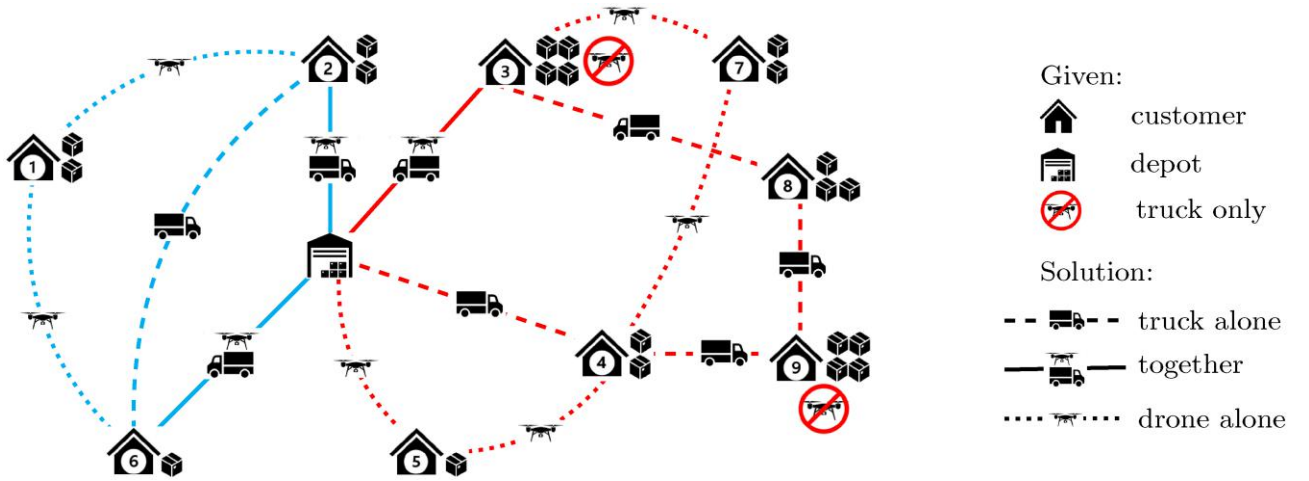
To distinguish between routing-cost and duration minimization, each edge $\{v, w\} \in E$ is associated with a nonnegative routing cost c_{vw} and a nonnegative travel time t_{vw} for a truck. Likewise, the routing cost of a drone is denoted by c_{vw}^{dr} and the travel time by t_{vw}^{dr} . The routing cost c_r of a route $r = (P, D)$ is defined as the sum of the routing costs of the edges traversed by both types of vehicles; that is, $c_r = \sum_{p=0}^{\ell} c_{v_p, v_{p+1}} + \sum_{s=1}^d (c_{v^s, k^s}^{\text{dr}} + c_{k^s, w^s}^{\text{dr}})$. The duration t_r of a route $r = (P, D)$ is more intricate to describe because the truck (drone) may have to wait for the drone (truck) depending on which vehicle is faster on a particular segment. Let $I = \bigcup_{s=1}^d I(s)$. Then, the duration of route r is

$$t_r = \sum_{p \in \{0, 1, \dots, \ell\} \setminus I} t_{v_p, v_{p+1}} + \sum_{s=1}^d \max \left\{ \sum_{p \in I(s)} t_{v_p, v_{p+1}}, t_{v^s, k^s}^{\text{dr}} + t_{k^s, w^s}^{\text{dr}} \right\}. \quad (1)$$

The first term describes travel times when truck and drone travel together, whereas the second term sums over the d drone subpaths taking the maximum of the travel time of either the truck or the drone.

The task of the VRP-D is to determine a set $\mathcal{R}$ of at most K feasible routes such that each customer is served exactly once, and either $\sum_{r \in \mathcal{R}} c_r$ or $\sum_{r \in \mathcal{R}} t_r$ is minimized.

Example 1. Figure 1 shows a VRP-D instance with customers $N = \{1, 2, \dots, 9\}$ (customers are shown as houses with numbers) as well as a feasible solution with two routes that both have a capacity of $Q = 20$.

Figure 1. (Color online) VRP-D Instance with a Feasible Solution

On the first route (depicted in red/on the right; we assume an anticlockwise traversal), the truck releases its drone directly at the depot. Whereas the truck travels to customer 4 and serves it, the drone moves to customer 5, serves this customer, and returns back to the truck at customer 4. Immediately at customer 4, truck and drone separate again: while the truck serves the customers 9, 8, and 3 alone, the drone serves customer 7 and lands on the truck again at the location of customer 3. Afterward, truck and drone together travel back to the depot. Hence, the route serves customers $\{3, 4, 5, 7, 8, 9\}$. The corresponding truck path is $P = (v_0, v_1, \dots, v_6) = (0, 4, 9, 8, 3, 0')$, and the sequence of drone subpaths is $D = (\langle 0, 5, 4 \rangle, \langle 4, 7, 3 \rangle)$. The boxes depicted beside the customers indicate the respective demand. The first route delivers 16 boxes and is therefore feasible.

The second route (depicted in blue/on the left; we assume a clockwise traversal) serves another three customers $\{1, 2, 6\}$ with $P = (0, 6, 2, 0')$ and $D = (\langle 6, 1, 2 \rangle)$. Assuming that all travel costs and travel times of all edges $\{v, w\} \in E$ are $c_{vw} = t_{vw} = c_{vw}^{\text{dr}} = t_{vw}^{\text{dr}} = 1$, the truck has to wait for the drone at location 4, and vice versa, the drone has to wait for the truck at location 3 in the first (red) route. Hence, the duration of the first route is $t_r = 6$, whereas its cost is $c_r = 9$. Under these assumptions, the truck has to wait for the drone at location 2 in the second (blue) route, which has a duration of $t_r = 4$, whereas its cost is $c_r = 5$. $\square$

4. Branch-Price-and-Cut Algorithm

We develop a BPC algorithm to solve the VRP-D. BPC algorithms are “the leading exact algorithms for solving many classes of vehicle routing problems” (see Costa, Contardo, and Desaulniers 2019). They rely on an

extensive route-based formulation (a.k.a. master problem (MP)), that is, a model with variables for all feasible routes. A BPC algorithm is a branch-and-bound algorithm in which, at each node, the linear relaxation of the MP is solved. At some nodes, valid inequalities are added to strengthen the linear relaxation. Because of the huge number of variables in the MP, relaxations are not solved directly but with a column-generation procedure: iteratively, *restricted master programs* (RMPs) defined over a subset of routes are solved, missing routes are identified by solving subproblems (a.k.a. pricing problems), and these are added to the RMPs (Desaulniers, Desrosiers, and Solomon 2005).

4.1. Route-Based Formulation

Let Ω denote the set of all feasible VRP-D routes. The route-based formulation presented below uses binary variables λ_r to indicate whether a route $r \in \Omega$ is part of the optimal solution. In addition, the nonnegative variable f describes the number of trucks employed. For each customer $v \in N$ and each route $r \in \Omega$, let the integer coefficient b_{vr} indicate the number of times the route r serves customer v .

$$\min \sum_{r \in \Omega} c_r \lambda_r \quad \text{or} \quad \min \sum_{r \in \Omega} t_r \lambda_r \quad (2a)$$

$$\text{subject to} \quad \sum_{r \in \Omega} b_{vr} \lambda_r = 1 \quad \forall v \in N \quad (2b)$$

$$\sum_{r \in \Omega} \lambda_r - f = 0 \quad (2c)$$

$$\underline{K} \leq f \leq K \quad (2d)$$

$$\lambda_r \in \{0, 1\} \quad \forall r \in \Omega. \quad (2e)$$

The Objective (2a) is to minimize the total routing cost or the sum of the durations of all routes. Constraints (2b) ensure that each customer is served exactly once.

The number of trucks f in use is determined via (2c) and restricted to the interval between a lower bound $\underline{K}$ and the given fleet size K via (2d). The domain of the route variables is specified in (2e).

A valid lower bound $\underline{K}$ for the number of necessary trucks can be calculated by solving a bin-packing problem as follows: The bin size is set equal to the truck's capacity Q . For each customer $i \in N$, an item with a weight equal to demand q_i must be introduced. We solve the bin-packing problem with the arc-flow formulation of Valério de Carvalho (1999) using an MIP solver. Note that the optimal number $\underline{K}$ of bins is an exact lower bound that helps to not solve some infeasible branch-and-bound nodes, which may otherwise result from branching on the number of vehicles (see Section 4.4). Solving these infeasible nodes is often very time-consuming.

4.2. Column Generation

Let $(\pi_v)_{v \in N} \in \mathbb{R}^{|N|}$ be the dual prices of the Set-Partitioning Constraints (2b) and $\mu \in \mathbb{R}$ be the dual price of Constraint (2c). The reduced cost of a route $r \in \Omega$ is $\tilde{c}_r^{\text{cost}} = c_r - \sum_{v \in N} b_{vr} \pi_v - \mu$ or $\tilde{c}_r^{\text{dur}} = t_r - \sum_{v \in N} b_{vr} \pi_v - \mu$ depending on whether the objective is cost or duration minimization.

The emerging pricing problem can be modeled as a *shortest path problem with resource constraints* (SPPRC) (Irnich and Desaulniers 2005) and solved by means of a dynamic-programming labeling algorithm on an artificial network.

4.2.1. Artificial Network. Compared with the classical CVRP, the major additional complication in the VRP-D is that truck and drone can move independently, at least on parts of their itinerary. Already the decision of which type of vehicle will serve which subset of the customers adds a substantial degree of freedom. All these decisions impact a route's cost and duration, which can no longer be computed as the length of a single path. We have shown in Section 3 that route feasibility constraints are much more involved compared with the CVRP because a route in the given network (V, E) consists of a truck path and a sequence of drone subpaths that have to fit together as defined in Section 3.

For solving the single-vehicle version of the problem, that is, the TSP-D, Roberti and Ruthmair (2021) model a route/tour with the help of an *artificial network* $N = (W, A)$. The vertex set $W = \{(0, 0)\} \cup (N \cup \{0'\}) \times (\{0\} \cup N) \cup \{(0', 0')\} \subset V \times V$ represents possible truck-and-drone positions; that is, the vertex $(i^{\text{tr}}, i^{\text{dr}}) \in W$ describes that the truck is at position i^{tr} , and the drone is at i^{dr} .

To lighten the notation, we assume in the following that writing $i \in W$ describes the vertex $(i^{\text{tr}}, i^{\text{dr}})$. When another vertex is needed, we write $j \in W$ to describe the vertex $(j^{\text{tr}}, j^{\text{dr}})$. Note that the symbols i and j will be exclusively used in this context.

In particular and with slight abuse of notation, the *origin* vertex $0 = (0, 0) \in W$ represents the initial situation at the depot when truck and drone are together ready to start a route. Likewise, the *destination* vertex $0' = (0', 0') \in W$ represents the final situation when truck and drone are back together at the depot after completing a route.

Arcs of the artificial network represent movements and a possible additional service of a customer by the drone. For the latter service aspect, let $N^{\text{dr}} = \{v \in N : q_v \leq \bar{q}\}$ denote such a service by drone, and let $\perp$ denote that no feasible service can be performed. The above convention to write i for $(i^{\text{tr}}, i^{\text{dr}})$ and j for $(j^{\text{tr}}, j^{\text{dr}})$ allows us to compactly describe an arc of $\mathcal{N}$ by triplets $[i, j, k]$ written out as $[(i^{\text{tr}}, i^{\text{dr}}), (j^{\text{tr}}, j^{\text{dr}}), k]$ (we use square brackets to distinguish arcs of the artificial network $\mathcal{N}$ from the edges $\{v, w\}$ of the original network (V, E) and from subpaths $\langle v, k, w \rangle$). The arcs of the artificial network $\mathcal{N}$ fall into the following three categories. First, the *alone arcs*

$$A^{\text{alone}} = \{[i, j, \perp] \in W \times W \times \{\perp\} : i^{\text{dr}} = j^{\text{dr}} \text{ and } i^{\text{tr}} \neq j^{\text{tr}} \text{ and } i^{\text{dr}} \neq j^{\text{tr}}\} \quad (3a)$$

represent that the truck is moving from position i^{tr} to position j^{tr} while the drone is airborne with launch position $i^{\text{dr}} = j^{\text{dr}}$. Second, the *together arcs*

$$A^{\text{together}} = \{[i, j, \perp] \in W \times W \times \{\perp\} : i^{\text{tr}} = i^{\text{dr}} \neq j^{\text{tr}} = j^{\text{dr}}\} \quad (3b)$$

represent that truck and drone are moving together from position $i^{\text{tr}} = i^{\text{dr}}$ to a new position $j^{\text{tr}} = j^{\text{dr}}$. In both cases, alone and together arcs, the customer j^{tr} is served. Third, the *drone arcs*

$$A^{\text{drone}} = \{[i, j, k] \in W \times W \times N^{\text{dr}} : i^{\text{tr}} = j^{\text{tr}} = j^{\text{dr}} \neq i^{\text{dr}} \text{ and } k \notin \{i^{\text{dr}}, i^{\text{tr}}\}\} \quad (3c)$$

represent that the drone flies from the position i^{dr} , where it has separated from the truck to customer k , serves customer k , and subsequently flies to the position $i^{\text{tr}} = j^{\text{tr}} = j^{\text{dr}}$, where the truck is waiting or will later arrive to pick up the drone. Note that the arc sets defined via (3a)–(3c) are disjoint. Hence, each arc in $A = A^{\text{alone}} \cup A^{\text{together}} \cup A^{\text{drone}}$ belongs to exactly one category (alone, together, or drone arc). Moreover, the arcs in $A^{\text{alone}} \cup A^{\text{together}}$ are referred to as *truck arcs* because they model serving customer j^{tr} by truck.

We stress that the artificial network $\mathcal{N}$ is directed and has parallel arcs; that is, it is a *multidigraph*. Parallel arcs are always drone arcs $[i, j, k] \in A^{\text{drone}}$, and they differ in the additional service that is performed at customer k .

The fundamental idea of Roberti and Ruthmair (2021) is to artificially separate truck movement decisions from drone movement decisions. When truck and drone separate, it is not necessary to know in advance which

customer the drone will serve and where and when it will return to the truck. Hence, these decisions can be postponed. Instead, the bookkeeping reduces to record the position where the drone has departed from the truck. The truck is then routed independently, serving an arbitrary number of customers. The postponed decision about the drone's subpath is made when truck and drone meet again. This enables a forward labeling algorithm with an efficient dominance procedure as shown in Section 4.2.2.

Example 2. Figure 2 visualizes the artificial network for a VRP-D instance with three customers $N = \{1, 2, 3\}$. The path $((0, 0), (1, 1), (2, 2), (3, 3), (0', 0'))$ in N represents a route where truck and drone travel together all the way from the depot 0 to the three customers 1, 2, and 3 (in this sequence) and back to the depot $0'$. Path $((0, 0), (2, 0), (2, 2), (3, 3), (0', 0'))$ in N has an associated unique truck path $(0, 2, 3, 0')$. However, there are two parallel arcs connecting $(2, 0)$ with $(2, 2)$, leaving the choice of using one of them. Either customer 1 could be served, leading to the associated drone subpath $\langle 0, 1, 2 \rangle$, or customer 3 could be served, leading to the associated drone subpath $\langle 0, 3, 2 \rangle$. In the first case, the route is elementary, serving customers 1, 2, and 3 exactly once, whereas in the second case, the route is nonelementary, serving customer 3 twice and customer 2 once. $\square$

More generally, every $0-0'$ -path in N , that is, a path from $(0, 0)$ to $(0', 0')$, represents the routes of a truck and its drone. For simplicity, we refer to them as *a route* in the following. Because the artificial network is a multidigraph, only the sequence of its arcs describes it uniquely. Note that the corresponding route is not necessarily elementary or feasible. The labeling algorithm presented in Section 4.2.2 checks elementary and feasibility.

Each $0-0'$ -path in N can be resolved into a corresponding truck path and drone subpaths using the information on each arc $[i, j, k]$. If the arc is a truck arc, that is, $k = \perp$, the truck destination j^{tr} has to be appended to the truck path. If the arc is a drone arc, that is, $k \in N^{\text{dr}}$, the drone subpath $\langle v, k, w \rangle$ contains the following three pieces of information: the release location $v = i^{\text{dr}}$ of the drone, the drone customer k , and the drone's destination location $w = j^{\text{dr}}$.

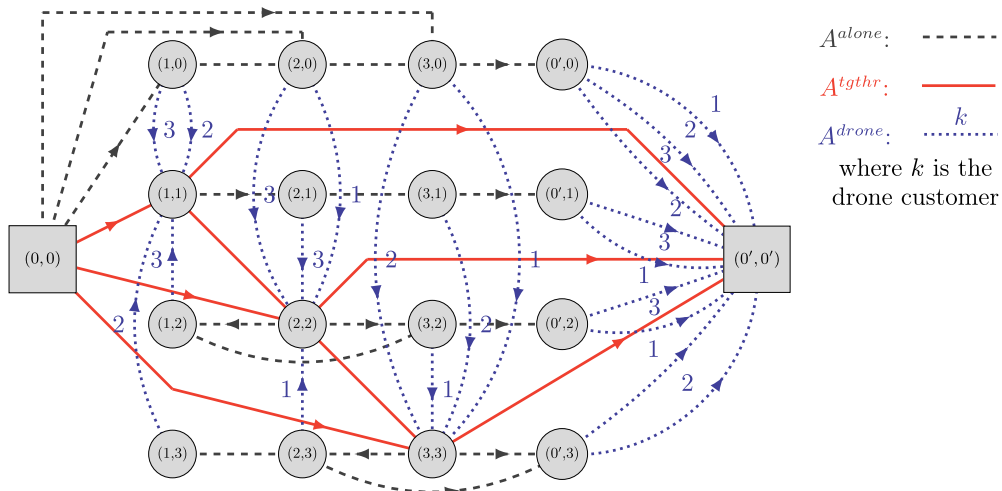
The following example introduces a second, probably more intuitive (graphical) notation for routes in the artificial network.

Example 3. We consider a VRP-D instance with $N = \{1, 2, 3, 4, 5\}$. The $0-0'$ -path $[(0, 0), (1, 1), \perp], [(1, 1), (2, 1), \perp], [(2, 1), (2, 2), 3], [(2, 2), (4, 2), \perp], [(4, 2), (0', 2), \perp], [(0', 2), (0', 0'), 5]$ has $P = (0, 1, 2, 4, 0')$ and $D = (\langle 1, 3, 2 \rangle, \langle 2, 5, 0' \rangle)$. In the following, we also use the more intuitive representation $(0, 0) = (1, 1) - (2, 1) \overset{3}{-} (2, 2) - (4, 2) - (0', 2) \overset{5}{-} (0', 0')$ as a sequence of vertices where the connections are either $=$, $-$, or $\overset{k}{-}$. The symbols $- (=)$ indicate a connection with an alone (together) arc, whereas the symbol $\overset{k}{-}$ indicates a drone arc of type $[(\cdot, \cdot), (\cdot, \cdot), k]$. $\square$

4.2.2. Forward Labeling. In this section, we present a dynamic-programming labeling algorithm to solve the SPPRC pricing problem. We begin with a basic forward labeling approach and then discuss refinements, including an implicit bidirectional labeling and a nontrivial merge procedure.

In our labeling algorithm, each label refers to a partial path $P_i = (0, \dots, i)$ that starts at the origin depot and ends at a vertex $i = (i^{\text{tr}}, i^{\text{dr}}) \in W$. The label stores the corresponding resource consumption up to the vertex i .

Figure 2. (Color online) Artificial Network $N = (W, A)$ with Customer Set $N = \{1, 2, 3\}$



Notes. For the sake of simplicity, not all truck arcs are made explicit. Antiparallel arcs between vertices are depicted as undirected links. For example, in the first row, antiparallel arcs exist between the vertices $(1, 0)$, $(2, 0)$, and $(3, 0)$.

The set of attributes (i.e., resources) depends on the objective function used in the VRP-D.

4.2.2.1. Cost Objective. For the cost objective and a partial path P_i , the associated label $\mathcal{L}_i$ is defined by the resource vector $R = (R_i^{rdc, cost}, R_i^{load}, (R_i^{cust, n})_{n \in N})$. In the following, we write in short $\mathcal{L} = (R)$ when referring to a label and its resource vector. The resources are defined as follows:

$R_i^{rdc, cost}$: the reduced cost of path P_i ;

R_i^{load} : the total demand served by path P_i excluding the demand of i ; and

$R_i^{cust, n}$: the number of times that customer $n \in N$ is served along the path P_i .

The initial label is given by $\mathcal{L}_0 = (0, 0, 0)$. Propagating a label $\mathcal{L}_i$ over an arbitrary arc $a = [i, j, k] \in A$ results in a new label $\mathcal{L}_j$ at vertex $j = (j^{tr}, j^{dr})$. We assume that dual prices $\pi = (\pi_v)_{v \in N}$ of the Constraints (2b) and the dual price μ of the Constraint (2c) are given. For convenience, we define $\pi_{0'} = \mu$ for the destination $0'$. Moreover, we define $q_0 = q_{0'} = 0$ for the depot. The associated resource values for the partial path $P_j = (P_i, j)$ are computed with the help of the following *resource extension functions* (REFs) (Irnich 2008):

$$R_j^{rdc, cost} = \begin{cases} R_i^{rdc, cost} + c_{j^{tr}, j^{tr}} - \pi_{j^{tr}}, & \text{if } [i, j, \perp] \in A^{alone} \cup A^{tgthr} \\ R_i^{rdc, cost} + c_{j^{dr}, k}^{dr} + c_{k, j^{dr}}^{dr} - \pi_k, & \text{if } [i, j, k] \in A^{drone} \end{cases} \quad (4a)$$

$$R_j^{load} = \begin{cases} R_i^{load} + q_{j^{tr}}, & \text{if } [i, j, \perp] \in A^{alone} \cup A^{tgthr} \\ R_i^{load} + q_k, & \text{if } [i, j, k] \in A^{drone} \end{cases} \quad (4b)$$

$$R_j^{cust, n} = \begin{cases} R_i^{cust, n} + 1, & \text{if } n = j^{tr} \text{ and } [i, j, \perp] \in A^{alone} \cup A^{tgthr} \\ R_i^{cust, n} + 1, & \text{if } n = k \text{ and } [i, j, k] \in A^{drone} \\ R_i^{cust, n}, & \text{otherwise.} \end{cases} \quad (4c)$$

Note that in (4b), the update of the load resource is postponed regarding the customer j^{tr} to the point after the truck leaves customer j^{tr} . We will see later that this is convenient for the symmetric bidirectional labeling algorithm.

The new label $\mathcal{L}_j$ is feasible if the capacity and the elementary constraints are fulfilled; that is,

$$R_j^{load} \leq \begin{cases} Q - q_{j^{tr}}, & \text{if } [i, j, \perp] \in A^{alone} \cup A^{tgthr} \\ Q, & \text{if } [i, j, k] \in A^{drone} \end{cases} \quad (5a)$$

$$R_j^{cust, n} \leq 1, \quad \forall n \in N. \quad (5b)$$

Infeasible labels are discarded immediately. To avoid the enumeration of all feasible paths, redundant labels are eliminated with the help of dominance tests. Because all forward REFs are nondecreasing, we can

use the standard dominance rules (Irnich and Desaulniers 2005):

Dominance Rule 1. We consider two labels at the same vertex of the artificial network. Label $\mathcal{L}_1 = (R_1^{rdc, cost}, R_1^{load}, (R_1^{cust, n})_{n \in N})$ dominates another label $\mathcal{L}_2 = (R_2^{rdc, cost}, R_2^{load}, (R_2^{cust, n})_{n \in N})$ if $R_1^{rdc, cost} \leq R_2^{rdc, cost}$, $R_1^{load} \leq R_2^{load}$, and $R_1^{cust, n} \leq R_2^{cust, n}$ holds for all $n \in N$.

4.2.2.2. Duration Objective. When considering the duration objective, additional information is required. We follow the ideas presented in Roberti and Ruthmair (2021). Hence, a label now comprises the resources $(R_i^{rdc, dur}, R_i^{dur}, R_i^{load}, (R_i^{cust, n})_{n \in N})$ defined as:

$R_i^{rdc, dur}$: the reduced cost based on the duration of path P_i ;

R_i^{dur} : the duration for which the truck travels alone on the last open subtour of path P_i , and R_i^{load} and $R_i^{cust, n}$ are as defined in (4b) and (4c), respectively. A subtour is an *open* subtour if one or several alone arcs have been traversed without closing the subtour with a drone arc.

The initial label is now given by $\mathcal{L}_0 = (0, 0, 0, 0)$. For an arc $[i, j, k] \in A$, the resources rdc , dur , and dur are updated by

$$R_j^{rdc, dur} = \begin{cases} R_i^{rdc, dur} - \pi_{j^{tr}}, & \text{if } [i, j, \perp] \in A^{alone} \\ R_i^{rdc, dur} + t_{j^{tr}, j^{tr}} - \pi_{j^{tr}}, & \text{if } [i, j, \perp] \in A^{tgthr} \\ R_i^{rdc, dur} & \text{if } [i, j, k] \in A^{drone} \\ + \max\{t_{j^{dr}, k}^{dr} + t_{k, j^{dr}}^{dr}, R_i^{dur}\} - \pi_k, & \end{cases} \quad (6a)$$

$$R_j^{dur} = \begin{cases} R_i^{dur} + t_{j^{tr}, j^{tr}}, & \text{if } [i, j, \perp] \in A^{alone} \\ 0, & \text{if } [i, j, \cdot] \in A^{drone} \cup A^{tgthr}. \end{cases} \quad (6b)$$

Compared with the cost objective, there are no further feasibility requirements in addition to (5).

Dominance Rule 2. We consider two labels at the same vertex $i = (i^{tr}, i^{dr})$ of the artificial network. Label $\mathcal{L}_1 = (R_1^{rdc, dur}, R_1^{dur}, R_1^{load}, (R_1^{cust, n})_{n \in N})$ dominates another label $\mathcal{L}_2 = (R_2^{rdc, dur}, R_2^{dur}, R_2^{load}, (R_2^{cust, n})_{n \in N})$ if $R_1^{rdc, dur} \leq R_2^{rdc, dur}$, $R_1^{dur} \leq R_2^{dur}$, $R_1^{load} \leq R_2^{load}$, and $R_1^{cust, n} \leq R_2^{cust, n}$ for all $n \in N$, and

- (1) $i^{tr} = i^{dr}$ and $R_1^{rdc, dur} \leq R_2^{rdc, dur}$,
- (2) or: $i^{tr} \neq i^{dr}$ and $R_1^{rdc, dur} \leq R_2^{rdc, dur}$, $R_1^{rdc, dur} + R_1^{dur} \leq R_2^{rdc, dur} + R_2^{dur}$,
- (3) or: $i^{tr} \neq i^{dr}$ and $R_1^{rdc, dur} + R_1^{dur} \leq R_2^{rdc, dur} + R_2^{dur}$ and $R_1^{rdc, dur} + \max_{k, v} \{t_{i^{dr}, k}^{dr} + t_{k, v}^{dr}\} \leq R_2^{rdc, dur} + R_2^{dur}$, where the indices k and v in the max-term run over $k \in N$ and $v \in N \cup \{0'\}$ with $k \neq v$.

Note that in part (1), the precondition $i^{tr} = i^{dr}$ implies $R_1^{dur} = R_2^{dur} = 0$ because of (6b), which proves correctness of the dominance because of nondecreasing REFs (4b), (4c), (6a), and (6b). In part (2), the condition $R_1^{rdc, dur} +$

$R_1^{dur} \leq R_2^{rdc,dur} + R_2^{dur}$ is fulfilled whenever $R_1^{rdc,dur} \leq R_2^{rdc,dur}$ and $R_1^{dur} \leq R_2^{dur}$, which shows that this dominance is stronger than a simpler pairwise comparison with $\leq$. The two conditions in part (3) guarantee that the reduced cost of the first path is not larger than that of the second when both are extended to a vertex j with $j^{tr} = j^{dr}$, either directly or with additional intermediate vertices. Such a vertex j with $j^{tr} = j^{dr}$ is certainly reached when arriving at the destination depot $0' = (0', 0')$. The validity of this Dominance Rule 2 is formally shown in Roberti and Ruthmair (2021).

4.2.3. Symmetry Considerations. By definition, the VRP-D is a symmetric routing problem, thus defined using an undirected graph. In this graph (V, E) , the two depot vertices 0 and $0'$ can be swapped without changing the VRP-D instance. By reversing a route's truck path and drone subpaths (if any) and by swapping 0 and $0'$ afterward, the resulting route is equivalent to the one considered originally. In particular, both routes serve the same customers and have identical routing cost and duration. In contrast, the artificial network $N = (W, A)$ is intentionally defined as a directed multigraph. This is consistent with the idea that the second component i^{dr} of a vertex $i \in W$ indicates the location where the truck has released the drone in case that $i^{dr} \neq i^{tr}$ holds. This piece of information is asymmetric because the choice of the drone's later landing location is unspecified. Even more, a given route r and the corresponding reversed route r' are represented, in general, by paths that visit different vertices in the artificial network N , as shown in the following example.

Example 4 (Continued from Example 3). The truck path $P = (0, 1, 2, 4, 0')$ and associated drone subpaths $D = (\langle 1, 3, 2 \rangle, \langle 2, 5, 0' \rangle)$ have reversed counterparts $P' = (0, 4, 2, 1, 0')$ and $D' = (\langle 0, 5, 2 \rangle, \langle 2, 3, 1 \rangle)$, which are symmetric in the original network (V, E) . In the artificial network, however, the perfectly corresponding $0-0'$ -paths $(0, 0) = (1, 1) \xrightarrow{3} (2, 2) \xrightarrow{4} (4, 2) \xrightarrow{5} (0', 2) \xrightarrow{5} (0', 0')$ and $(0, 0) = (4, 0) \xrightarrow{5} (2, 0) \xrightarrow{2} (1, 2) \xrightarrow{3} (1, 1) \xrightarrow{4} (0', 0')$ differ in the vertices $(1, 2)$, $(2, 1)$, $(4, 2)$, $(4, 0)$, $(0', 2)$, and $(2, 0)$. $\square$

The example shows that the reversal of paths in the asymmetric network is somewhat confusing and not straightforward. It seems rather unpromising to develop a bidirectional labeling algorithm over the artificial network. These observations may explain why Roberti and Ruthmair (2021) did not present a bidirectional approach for the TSP-D, although the general methodology for an exact solution algorithm for TSP variants is known (Baldacci, Mingozzi, and Roberti 2012, Tilk and Irnich 2017, Lera-Romero, Miranda Bront, and Soullignac 2022).

4.2.4. Implicit Backward and Bidirectional Labeling. For many SPPRCs, the number of labels generated by a

monodirectional labeling algorithm increases rapidly when feasible partial paths grow longer. To mitigate this so-called *combinatorial explosion*, Righini and Salani (2006) introduced a bounded bidirectional labeling procedure for SPPRCs. In a bidirectional labeling, not only forward labels but also backward labels are created by propagating a label against the orientation of the arcs starting from the sink of the network. Both forward and backward labels are only extended up to a so-called *halfway point* (HWP). When forward and backward propagation is completed, suitable labels are merged to obtain complete feasible origin-destination paths.

The selection of a monotone resource (often the load or time attribute) and the concrete choice of the HWP not exactly “in the middle” can be important for the labeling algorithm's performance for asymmetric SPPRC instances (Tilk et al. 2017). If the underlying network is, however, perfectly symmetric, an *implicit* bidirectional technique can be applied to further accelerate the labeling. Because forward and backward labeling create exactly the same partial paths, it suffices to propagate labels only in the forward direction. These labels are combined in the merge procedure. Implicit bidirectional labeling has already been applied successfully in several works (e.g., Bode and Irnich 2012, Goeke, Gschwind, and Schneider 2019, Heßler and Irnich 2023).

We now show that implicit bidirectional labeling is possible for the artificial network of the VRP-D, although it is asymmetric. Because the only constrained resource is the capacity of the trucks, we use the load resource R^{load} as the critical resource and fix the HWP to $Q/2$; that is, only labels with accumulated load $R^{load} < Q/2$ are extended.

The following observation can be made from Example 4: The two $0-0'$ -paths that represent reversed routes differ only in vertices $i \in W$ with $i^{tr} \neq i^{dr}$. These vertices occur in segments of the routes where truck and drone move independently. In contrast, when truck and drone are at the same location $i^{tr} = i^{dr}$, segments are identical. In Example 4, the sequence $(0, 0)$, $(1, 1)$, $(2, 2)$, $(0', 0')$ is traversed in either this or the reversed order.

As a consequence, we apply two different merge procedures depending on whether $i^{tr} = i^{dr}$ or $i^{tr} \neq i^{dr}$ holds at the vertex i of a label that could potentially be merged. The procedure for $i^{tr} = i^{dr}$ is rather standard, whereas the one for $i^{tr} \neq i^{dr}$ is more sophisticated.

4.2.4.1. Merge at Vertices Where Truck and Drone Are Together ($i^{tr} = i^{dr}$). In this case, we consider two labels, $\mathcal{L} = (R)$ and $\mathcal{L}' = (R')$, of partial paths that both end at the same vertex i with $i^{tr} = i^{dr}$ (this is the so-called *merge vertex*). These vertices exist for all $i^{tr} \in V$. The merge condition is the one known from the CVRP; that is, the resulting path must respect the truck's capacity

and be elementary:

$$R^{load} + R'^{load} + q_{i^{tr}} \leq Q \quad (7a)$$

$$R^{cust,n} + R'^{cust,n} \leq 1 \quad \forall v \in N \setminus \{i^{tr}\}. \quad (7b)$$

The reduced cost of route r resulting from the merge is

$$\begin{aligned} \tilde{c}_r^{cost} &= R^{rdc, cost} + R'^{rdc, cost} + \pi_{i^{tr}} \quad \text{and} \\ \tilde{c}_r^{dur} &= R^{rdc, dur} + R'^{rdc, dur} + \pi_{j^{tr}}. \end{aligned} \quad (8a)$$

The route itself is $r = (P, \text{rev}^-(P'))$, where P and P' are the partial paths associated with $\mathcal{L}$ and $\mathcal{L}'$, respectively, and the function $\text{rev}^-(P')$ denotes the reversed path to P' (in the sense of Example 4, also replacing 0 by 0'; details are provided below) from which the last vertex is removed (to avoid including the merge vertex twice).

Even if we do not need to know the reversed path for checking the merge conditions and computing the reduced cost, we must formally describe the reversal of the second partial path P' associated with $\mathcal{L}'$. For the reversal of an arbitrary path, it suffices to describe the reversal of a *segment* between two subsequent vertices i and j with $v = i^{tr} = i^{dr}$ and $w = j^{tr} = j^{dr}$. In general, such a segment in the artificial network is either trivial; that is, it consists of a single together arc, or it is of the form $(v, v) - (v_1, v) - (v_2, v) - \dots - (v_l, v) - (w, v) \xrightarrow{k} (w, w)$ with associated drone subpath $\langle v, k, w \rangle$. In this segment, all arcs are alone arcs except for the last arc, which is the drone arc $[(w, v), (w, w), k]$. The reversed segment is $(w, w) - (v_l, w) - \dots - (v_2, w) - (v_1, w) - (v, w) \xrightarrow{k} (v, v)$; that is, (1) the sequence $v_1, v_2, \dots, v_l$ of truck positions is reversed in the usual way, whereas the drone position v in the forward segment is replaced by w for all vertices in the sequence except the first and last vertex; and (2) the second-last vertex (w, v) in the forward segment becomes the second last vertex (v, w) in the reversed segment. Note also that in both segments, the customer k is served by the drone; that is, the last arc in the reversed path is the drone arc $[(v, w), (v, v), k]$. Finally, multiple reversed segments must be concatenated again in the usual way.

Example 5 (Continued from Example 4). We assume that the path $(0, 0) = (1, 1) - (2, 1) \xrightarrow{3} (2, 2) - (4, 2) - (0', 2) \xrightarrow{5} (0', 0')$ considered in Example 4 is feasible; that is, $\sum_{l=1}^5 q_l \leq Q$ is fulfilled.

Let vertex $(2, 2)$ be the merge vertex for the path; hence, it must hold $q_1 < Q/2$ and $q_1 + q_3 \geq Q/2$. In this case, the merge of the two partial paths $(0, 0) = (1, 1) - (2, 1) \xrightarrow{3} (2, 2)$ and $(0, 0) - (4, 0) - (2, 0) \xrightarrow{5} (2, 2)$ produces the given path. Note that these are exactly the partial paths (up to vertex $(2, 2)$) of the two partial paths shown in Example 4.

Their labels $\mathcal{L} = (R)$ and $\mathcal{L}' = (R')$ qualify for the merge because $R^{load} = q_1 + q_3 \geq Q/2$. The merge conditions are also fulfilled; what can be seen follows. First, the load-related Feasibility Test (7a) adds together $R^{load} = q_1 + q_3$, $R'^{load} = q_4 + q_5$ and q_2 for the merge

vertex $(2, 2)$, which is exactly the demand delivered by the resulting route. The assumption that the given route is feasible implies that (7a) is fulfilled.

Second, regarding the served customers, $R^{cust,n} = 1$ if and only if $n \in \{1, 2, 3\}$, and $R'^{cust,n} = 1$ if and only if $n \in \{2, 4, 5\}$. The Elementarity Check (7b) excludes $n = 2$ from the test $R^{cust,n} + R'^{cust,n} \leq 1$. Hence, labels $\mathcal{L}$ and $\mathcal{L}'$ pass all merge conditions. $\square$

Example 5 also provides a good example of why we must delay the addition of the customer demand in the REF (4b). In an alternative model, the demand would always be immediately added; that is, $q_{j^{tr}}$ were added to R^{load} on all arcs $[(i^{tr}, i^{dr}), (j^{tr}, j^{dr}), \perp]$ with $i^{tr} \neq j^{tr}$. Then, the partial paths of both labels R and R' could sooner pass the HWP, in which case they would not be extended to the merge point.

Example 6 (Continued from Example 5). We consider an instance of the VRP-D with demands $q_1 = q_2 = q_4 = 10$ and $q_3 = q_5 = 1$ and a vehicle capacity of $Q = 38$ as well as the same paths as in Example 5. Moreover, we assume a modified REF for the load resource that directly incorporates the demand of a new customer visited by the truck.

The first path $(0, 0) = (1, 1) - (2, 1) \xrightarrow{3} (2, 2)$ would pass the HWP $Q/2 = 19$ already at vertex $(2, 1)$ with an accumulated demand of $q_1 + q_2 = 20 > Q/2$. Similarly, the second path $(0, 0) - (4, 0) - (2, 0) \xrightarrow{5} (2, 2)$ would pass the HWP already at vertex $(2, 0)$ with an accumulated demand of $q_4 + q_2 = 20 > Q/2$. As a result, the merge procedure would not generate the route that has the two partial paths as forward and (reversed) backward partial paths. $\square$

4.2.4.2. Merge at Pairs of Vertices Where Truck and Drone Are at Different Locations ($i^{tr} \neq i^{dr}$).

This second case is more intricate: A label belonging to a vertex (i^{tr}, i^{dr}) with $i^{tr} \neq i^{dr}$ must not be merged with labels belonging to the same vertex. Instead, the second label must belong to a vertex with identical truck position i^{tr} but a different drone position $j^{dr} \neq i^{dr}$, that is, a vertex of the type (i^{tr}, j^{dr}) . As a consequence, the merge must be performed “over an arc” (and (i^{tr}, i^{dr}) and (i^{tr}, j^{dr}) are the so-called merge vertices) so that the total length of the forward and backward partial paths is by one arc smaller than the length of the resulting merged route.

Example 7 (continued from Example 5). We consider the same route as in Example 5 but assume that the merge involves the forward and backward partial paths $(0, 0) = (1, 1) - (2, 1) \xrightarrow{3} (2, 2) - (4, 2)$ and $(0, 0) - (4, 0)$; that is, the first (forward) partial path with label $\mathcal{L}$ ends at vertex $(i^{tr}, i^{dr}) = (4, 2)$, and the second (backward) partial path with label $\mathcal{L}'$ ends at vertex $(i^{tr}, j^{dr}) = (4, 0)$. Note that, compared with Example 5, the forward path is one arc longer, but the backward path is two arcs shorter.

Clearly, the merged truck path (in forward direction) is $(0, 1, 2, 4, 0')$. The first partial path has two drone subpaths $\langle 1, 3, 2 \rangle$ and $\langle 2, \cdot, \cdot \rangle$, where the latter is incomplete. Likewise, the second partial path has an incomplete subpath $\langle 0, \cdot, \cdot \rangle$. The two incomplete subpaths can be combined into one subpath $\langle 2, \cdot, 0' \rangle$ (by reversing $\langle 0, \cdot, \cdot \rangle$). For the drone subpath to be valid, a drone customer k^* needs to be chosen from the set N so that it does not conflict with the elementarity constraints. $\square$

The example shows that an appropriate drone customer k must be chosen when merging two partial paths with an incomplete (last) drone subpath. Let label $\mathcal{L} = (R)$ represent a partial path ending at $i \in W$ with $i^{\text{tr}} \neq i^{\text{dr}}$ and label $\mathcal{L}' = (R')$ be a partial path ending at $j \in W$ with $j^{\text{tr}} = i^{\text{tr}}$ and $j^{\text{dr}} \neq i^{\text{dr}}$. When merging $\mathcal{L}$ and $\mathcal{L}'$, the completed drone subpath is $\langle i^{\text{dr}}, k, j^{\text{dr}} \rangle$ for a customer $k \in N$ served on the additional drone arc. The reduced cost of the resulting route r is

$$\tilde{c}_r^{\text{cost}} = R^{\text{rdc}, \text{cost}} + R'^{\text{rdc}, \text{cost}} + \pi_{i^{\text{tr}}} + c_{i^{\text{dr}}, k}^{\text{dr}} + c_{k, j^{\text{dr}}}^{\text{dr}} - \pi_k \quad \text{or} \quad (8b)$$

$$\tilde{c}_r^{\text{dur}} = R^{\text{rdc}, \text{dur}} + R'^{\text{rdc}, \text{dur}} + \pi_{i^{\text{tr}}} + \max\{R^{\text{dur}} + R'^{\text{dur}}, t_{i^{\text{dr}}, k}^{\text{dr}} + t_{k, j^{\text{dr}}}^{\text{dr}}\} - \pi_k, \quad (8c)$$

depending on the objective, that is, either cost or duration minimization. The route is feasible if

$$R^{\text{load}} + R'^{\text{load}} + q_{i^{\text{tr}}} + q_k \leq Q \quad (9a)$$

$$R^{\text{cust}, n} + R'^{\text{cust}, n} \leq \begin{cases} 1, & n \in N \setminus \{i^{\text{tr}}, k\} \\ 0, & n = k. \end{cases} \quad (9b)$$

Accordingly, to obtain a minimum reduced-cost route, the drone customer k^* is chosen as

$$k^* = \underset{k \in N(R, R', i^{\text{tr}})}{\operatorname{argmin}} \{c_{i^{\text{dr}}, k}^{\text{dr}} + c_{k, j^{\text{dr}}}^{\text{dr}} - \pi_k\} \quad \text{or} \\ k^* = \underset{k \in N(R, R', i^{\text{tr}})}{\operatorname{argmin}} \{\max\{R^{\text{dur}} + R'^{\text{dur}}, t_{i^{\text{dr}}, k}^{\text{dr}} + t_{k, j^{\text{dr}}}^{\text{dr}}\} - \pi_k\}, \quad (10)$$

where the subset of the customers to choose from is defined as $N(R, R', i^{\text{tr}}) = \{k \in N : (9) \text{ is fulfilled}\}$. For convenience, we assume that the minimum is ∞ for $N(R, R', i^{\text{tr}}) = \emptyset$. All routes with $\tilde{c}_r \geq 0$ are rejected in the merge procedure.

Finally, to avoid the generation of the same route multiple times, we restrict the merge to the cases when either $R^{\text{load}} > Q/2$ or $R'^{\text{load}} > Q/2$ or $i^{\text{tr}} = 0'$ holds true.

4.2.5. ng-Route Relaxation. The elementarity constraints are those constraints that make many SPPRCs not only theoretically but also practically difficult to solve. The use of an SPPRC relaxation offers an easier solution of the pricing problems but comes at the cost of a generally weaker linear relaxation of the corresponding MP. For the CVRP and the VRP with time windows

(VRPTW), Baldacci, Mingozzi, and Roberti (2011) introduced the *ng*-route relaxations to gradually control the trade-off between the strength of the LP relaxation and the practical difficulty of the SPPRC pricing problems. The concrete *ng*-route relaxation is defined by neighborhoods $N_i \subseteq N$, one for each vertex i . A cycle $(i, i_1, \dots, i_\ell, i)$ over vertex i (with $\ell \geq 1$) is feasible in the relaxation if at least one vertex i_p (for some $p \in \{1, 2, \dots, \ell\}$) fulfills $i \notin N_{i_p}$.

The *ng*-route relaxation is not directly applicable in the artificial network of the VRP-D. We adapt it (similar to Roberti and Ruthmair 2021) as follows:

- For each customer $v \in V$, we define a neighborhood $N_v \subseteq N$ based on the n_{ng} customers closest to v (the parameter n_{ng} controls the size of the neighborhoods and, herewith, the strength and difficulty of the relaxation).

- We equip each artificial vertex $(v, w) \in W$ with the neighborhood N_v ; that is, the service tasks in the neighborhood depend only on the truck vertex.

- In particular, for $i^{\text{tr}} \in N$, the neighborhood $N_{(i^{\text{tr}}, i^{\text{dr}})}$ contains customer i^{tr} .

The REF (4c) is altered by setting $R_j^{\text{cust}, n} = 0$ if $n \notin N_j$. The BPC implementation used in the computational experiments presented in Section 5 uses $n_{\text{ng}} = 3$ and extends the neighborhoods dynamically (see below). Note that the merge conditions $R^{\text{cust}, n} + R'^{\text{cust}, n} \leq 1$ and (9b) remain valid.

Monodirectional (forward) labeling and bidirectional labeling have slightly different *ng*-route relaxations even if the neighborhoods are chosen identically; the monodirectional relaxation can be weaker. The effect is observed for routes with two drone subpaths serving the same drone customer k . Such a nonelementary route can be feasible in the *ng*-route relaxation when labeling is performed completely in the forward direction, but the same route can be excluded in the merge.

Example 8. We consider a route with truck path $(0, 1, 2, 3, 0')$ and two drone subpaths $\langle 0, 4, 2 \rangle$ and $\langle 2, 4, 0' \rangle$; that is, the drone customer $k = 4$ is served twice. In the artificial network $N = (W, A)$, the associated route is $(0, 0) - (1, 0) - (2, 0) \stackrel{4}{\leftarrow} (2, 2) - (3, 2) - (0', 2) \stackrel{4}{\leftarrow} (0', 0')$. If the neighborhood of vertex 3 or $0'$ does not include $k = 4$, this route is feasible in the forward labeling algorithm.

For the bidirectional case, we assume that the merge happens at vertex $(i^{\text{tr}}, i^{\text{dr}}) = (2, 2)$ and that vertex 2 has $k = 4$ included in its neighborhood. Then, the two partial paths are $(0, 0) - (1, 0) - (2, 0) \stackrel{4}{\leftarrow} (2, 2)$ and $(0, 0) - (3, 0) - (2, 0) \stackrel{4}{\leftarrow} (2, 2)$ and are both *ng*-feasible with resource values $R^{\text{cust}, 4} = R'^{\text{cust}, 4} = 1$ at the merge vertex $(2, 2)$. However, the merge condition for the case $i^{\text{tr}} = i^{\text{dr}}$ requires (7b); that is, $R^{\text{cust}, 4} + R'^{\text{cust}, 4} \leq 1$, which is clearly violated. Hence, the considered route is excluded in the implicit bidirectional labeling. $\square$

Finally, note that inconsistent results would be observed if the halfway point would be chosen

dynamically (Tilk et al. 2017). However, implicit bidirectional labeling requires a fixed halfway point exactly in the middle at $Q/2$.

4.2.5.1. Dynamic Neighborhood Extension (DNE).

DNE is a technique to strengthen *ng*-route relaxations that has been independently proposed by Roberti and Mingozzi (2014) and Bode and Irnich (2015). Neighborhoods N_i are extended depending on the solutions obtained in the MP. We adapt DNE as follows. When the root node of the branch-and-bound tree is solved, we check whether routes of the MP solution serve customers more than once (each double service creates a *task cycle*; see also Irnich and Desaulniers 2005, section 2.2). For each elementary task cycle, we determine its flow value, that is, the sum of the λ_r^* -values over all routes r that contain the task cycle. We then choose the customer w^* and the task cycle Z^* that maximize the flow value divided by the length of the task cycle. For this cycle $Z^* = (w^*, v_1^*, \dots, v_\ell^*, w^*)$, the *ng*-neighborhoods $N_{v_1^*}, \dots, N_{v_\ell^*}$ are extended by w^* . This eliminates the task cycle Z^* in subsequent solutions to the pricing problem. Furthermore, the corresponding routes have to be removed from the RMP. After extending the neighborhood, the root node is solved again, and the check for task cycles is repeated for the resulting solution. We stop this procedure when all routes are elementary or 50 task cycles have been eliminated.

4.2.6. Acceleration Techniques. We now describe exact and heuristic acceleration techniques for the column-generation process.

4.2.6.1. Acceleration of the Merge Procedure. The merge procedure is the bottleneck of the column-generation process. In comparison with classical VRPs (like CVRP and VRPTW), the artificial network contains a quadratic number $\mathcal{O}(m^2)$ of vertices, and for those vertices i with $i^{\text{tr}} \neq i^{\text{dr}}$, the merge must be performed “over arcs.” All these arcs are drone arcs $[i, j, k]$ with $i^{\text{tr}} = j^{\text{tr}}$, $i^{\text{dr}} \neq j^{\text{dr}}$, and $k \in N$. As a result, the number of arcs to consider in the merge is $\mathcal{O}(m^4)$. This explains the empirical observation that the merge is very time-consuming. Therefore, we stop the merge procedure as soon as 100 negative reduced-cost routes are found (we use the same termination condition in the forward labeling algorithm).

To reduce the computational effort of the merge procedure further, we apply two exact acceleration techniques. First, when the forward labeling process is finished, all labels $\mathcal{L}$ belonging to the same vertex $(i^{\text{tr}}, i^{\text{dr}})$ are sorted in nondecreasing order by their load resource R^{load} . If truck and drone are at the same position, that is, both forward and backward labels belong to a vertex i with $i^{\text{tr}} = i^{\text{dr}}$, the capacity constraint $R^{\text{load}} + R'^{\text{load}} + q_{i^{\text{tr}}} \leq Q$ can be exploited as follows. In two nested loops running over candidate labels R and R' with $R^{\text{load}} \leq R'^{\text{load}}$, the

inner loop for R' can be discontinued as soon as $R^{\text{load}} + R'^{\text{load}} + q_{i^{\text{tr}}} > Q$. Moreover, the outer loop for R can be discontinued as soon as $2R^{\text{load}} + q_{i^{\text{tr}}} > Q$. If truck and drone are at different positions, then we know that the backward label must belong to another vertex j with $j^{\text{tr}} = i^{\text{tr}}$ and $j^{\text{dr}} \neq i^{\text{dr}}$. In this case, the capacity constraint can be further exploited: the load onboard is not smaller than $R^{\text{load}} + R'^{\text{load}} + q_{i^{\text{tr}}} + \min_{k \in N} \{q_k\}$. Hence, when considering the two nested loops, the outer over labels R at vertex $(i^{\text{tr}}, i^{\text{dr}})$ and the inner over labels R' at vertex $(i^{\text{tr}}, j^{\text{dr}})$, the inner loop can be discontinued if the above value exceeds Q .

Second, the determination of a best drone customer in (10) is another time-consuming task that has to be done numerous times in the merge procedure. To reduce the computational burden, we precompute the reduced cost for each possible drone subpath $\langle i^{\text{dr}}, k, j^{\text{dr}} \rangle$ before the actual procedure starts. For each combination $(i^{\text{dr}}, j^{\text{dr}}) \in V \times V$, the reduced cost of the drone subpath (i.e., $c_{i^{\text{dr}}, k}^{\text{dr}} + c_{k, j^{\text{dr}}}^{\text{dr}} - \pi_k$ or $t_{i^{\text{dr}}, k}^{\text{dr}} + t_{k, j^{\text{dr}}}^{\text{dr}} - \pi_k$) is stored in a list together with the respective drone customer k . The list is then sorted in nondecreasing order by reduced cost. When using the cost objective, a drone customer $k \in N(R, R', i^{\text{tr}}) \subset N$ for a merge with minimum reduced cost is found by inspecting the sorted list until a feasible merge is found (one that fulfills (9)). Moreover, the loop over $k \in N(R, R', i^{\text{tr}})$ can be stopped as soon as reduced cost can be estimated as being nonnegative. When using the duration objective, however, the reduced cost of the merge also depends on the sum $R^{\text{dur}} + R'^{\text{dur}}$, and we cannot determine a merge with minimum reduced cost as the first one in the sorted list that is feasible. Moreover, the loop over $k \in N(R, R', i^{\text{tr}})$ cannot be stopped prematurely. However, inspecting potential drone customers in sorted ordering helps to find a negative reduced-cost merge sooner, which accelerates the pricing also slightly in this case.

4.2.6.2. Partial Pricing with Reduced Networks. We use a hierarchy of heuristics to quickly identify negative reduced-cost routes. The use of one or more heuristics is also known as *partial pricing* (Gamache et al. 1999). In the context of dynamic programming-based labeling, partial pricing results from using the bidirectional labeling algorithm applied to an appropriate subgraph of the artificial network $N = (W, A)$. For the VRP-D, we use two reduced networks. The first has up to four ingoing and four outgoing truck arcs ($A^{\text{alone}} \cup A^{\text{tgthr}}$), as well as at most two drone arcs (A^{drone}) per artificial vertex. The second reduced network doubles the number of arcs per vertex (eight and four). To build the networks, the arcs with the smallest reduced cost in the current pricing iteration are selected. Pricing always starts with the first network, and the pricing problem is solved exactly only when both heuristics fail to provide any routes with negative reduced cost.

4.3. Valid Inequalities

We strengthen the linear relaxation of Formulation (2) with the help of *subset-row inequalities* (SRIs) (Jepsen et al. 2008) and nonrobust *capacity cuts* (CCs) (Baldacci, Christofides, and Mingozzi 2008).

A SRI is defined for a subset $S \subset N$ of customers and nonnegative weights for each $v \in S$. We restrict ourselves to subsets of size three and weight $1/2$ because these can be easily separated by straightforward enumeration and handled with a single binary attribute per active SRI in the labeling. The valid inequality reads

$$\sum_{r \in \Omega} \left\lfloor \sum_{v \in S} \frac{b_{vr}}{2} \right\rfloor \lambda_r \leq 1. \quad (11)$$

A CC is also defined for a subset $C \subset N$ of customers. Let K_C be a lower bound on the number of routes needed to serve C . The valid inequality for (C, K_C) is

$$\sum_{r \in \Omega} \max\{\min\{1, b_{vr}\}\} \lambda_r \geq K_C, \quad (12)$$

where we use the easy-to-compute lower bound $K_C = \lceil \sum_{v \in C} q_v / Q \rceil$ (the exact solution of a bin-packing problem like in Section 4.1 would sometimes create stronger inequalities at the cost of a much more time-consuming separation procedure).

Let $\mathcal{S}$ and $\mathcal{C}$ denote the sets of active SRIs and CCs, respectively. Further, let $\sigma_S < 0$ be the dual price of the SRI defined by $S \in \mathcal{S}$, and let $\tau_C > 0$ be the dual price of the CC defined by $(C, K_C) \in \mathcal{C}$. We define the additional binary resources $(R^{sri, S})_{S \in \mathcal{S}}$ and $(R^{cc, C})_{(C, K_C) \in \mathcal{C}}$. Initially, they are set to $(R^{sri, S}) = \mathbf{0}$ and $(R^{cc, C}) = \mathbf{0}$. For every second time a partial path serves a customer in S , the dual price σ_S has to be subtracted from the reduced cost. Similarly, the first time when a partial path serves a customer in C , the dual price τ_C has to be subtracted from the reduced cost. As for the load resource, it is important for a correct bidirectional labeling to delay the manipulation of the resource values when propagating over an arc $a = [i, j, k] \in A$. This is modeled in the following way:

$$R_j^{rdc, cost} = \begin{cases} R_i^{rdc, cost} + c_{i^tr, j^tr} - \pi_{j^tr} - \sum_{\substack{S \in \mathcal{S}: R_i^{sri, S} = 1, \\ i^tr \in S}} \sigma_S - \sum_{\substack{C \in \mathcal{C}: R_i^{cc, C} = 0, \\ i^tr \in C}} \tau_C, & \text{if } [i, j, \perp] \in A^{alone} \cup A^{tghr} \\ R_i^{rdc, cost} + c_{i^tr, k}^{dr} + c_{k, j^tr}^{dr} - \pi_k - \sum_{\substack{S \in \mathcal{S}: R_i^{sri, S} = 1, \\ k \in S}} \sigma_S - \sum_{\substack{C \in \mathcal{C}: R_i^{cc, C} = 0, \\ k \in C}} \tau_C, & \text{if } [i, j, k] \in A^{drone} \end{cases} \quad (4a')$$

$$R_j^{sri, S} = \begin{cases} 1 - R_i^{sri, S}, & \text{if } ([i, j, \perp] \in A^{alone} \cup A^{tghr} \text{ and } i^tr \in S) \\ & \text{or } ([i, j, k] \in A^{drone} \text{ and } k \in S) \\ R_i^{sri, S}, & \text{otherwise} \end{cases}$$

$$R_j^{cc, C} = \begin{cases} 1, & \text{if } ([i, j, \perp] \in A^{alone} \cup A^{tghr} \text{ and } i^tr \in C) \\ & \text{or } ([i, j, k] \in A^{drone} \text{ and } k \in C) \\ R_i^{cc, C}, & \text{otherwise.} \end{cases}$$

The new Equation (4a') for the update of the reduced cost replaces the original REF (4a) for the cost-minimization objective. Likewise,

$$R_j^{rdc, dur} = \begin{cases} R_i^{rdc, dur} - \pi_{j^tr} - \sum_{\substack{S \in \mathcal{S}: R_i^{sri, S} = 1, \\ i^tr \in S}} \sigma_S - \sum_{\substack{C \in \mathcal{C}: R_i^{cc, C} = 0, \\ i^tr \in C}} \tau_C, & \text{if } [i, j, \perp] \in A^{alone} \\ R_i^{rdc, dur} + t_{i^tr, j^tr} - \pi_{j^tr} - \sum_{\substack{S \in \mathcal{S}: R_i^{sri, S} = 1, \\ i^tr \in S}} \sigma_S - \sum_{\substack{C \in \mathcal{C}: R_i^{cc, C} = 0, \\ i^tr \in C}} \tau_C, & \text{if } [i, j, \perp] \in A^{tghr} \\ R_i^{rdc, dur} + \max\{t_{i^tr, k}^{dr} + t_{k, j^tr}^{dr}, R_i^{dur}\} & \text{if } [i, j, k] \in A^{drone} \\ - \pi_k - \sum_{\substack{S \in \mathcal{S}: R_i^{sri, S} = 1, \\ k \in S}} \sigma_S - \sum_{\substack{C \in \mathcal{C}: R_i^{cc, C} = 0, \\ k \in C}} \tau_C, & \end{cases} \quad (6a')$$

replaces the original REF (6a) for the duration objective. There are no feasibility constraints associated with new resources. The Dominance Rules 1 and 2 have to be modified in the standard way as explained in Baldacci, Christofides, and Mingozzi (2008) and Jepsen et al. (2008). Moreover, in the merge procedure, Reduced Cost Computations (8a) and (8b) have to be altered. For the merge of two labels R and R' at the same vertex i with $i^tr = i^{dr}$, the reduced cost becomes $\tilde{c}_r^{cost} = R^{rdc, cost} + R'^{rdc, cost} + \pi_{i^tr} + (*)$ and $\tilde{c}_r^{dur} = R^{rdc, dur} + R'^{rdc, dur} + \pi_{i^tr} + (*)$, where

$$(*) = \sum_{\substack{C \in \mathcal{C}: \\ R^{cc, C} = R'^{cc, C} = 1}} \tau_C - \sum_{\substack{S \in \mathcal{S}: \\ R^{sri, S} = R'^{sri, S} = 1}} \sigma_S - \sum_{\substack{S \in \mathcal{S}: i \in S \\ R^{sri, S} + R'^{sri, S} = 1}} \sigma_S \\ - \sum_{\substack{C \in \mathcal{C}: i \in C \\ R^{cc, C} = R'^{cc, C} = 0}} \tau_C$$

(the first sum corrects dual prices of CCs whose customers have been visited in both partial paths, the second sum incorporates the dual prices of SRIs with an odd number of visits in both partial paths, the third incorporates dual prices of SRIs with $i \in S$, and the last sum incorporates CCs containing customer i that were not visited by both partial paths).

For the merge of a label R at vertex i with $i^tr \neq i^{dr}$ and a label R' at vertex j with identical truck vertex $j^tr = i^tr$ but different drone vertex $j^{dr} \neq j^tr$ serving customer $k \in N$ on the drone arc, the reduced cost becomes

$$\tilde{c}_r^{cost} = R^{rdc, cost} + R'^{rdc, cost} + \pi_{i^tr} + c_{i^tr, k}^{dr} + c_{k, j^tr}^{dr} - \pi_k \\ - \sum_{\substack{S \in \mathcal{S}: k \in S, i^tr \notin S \\ R^{sri, S} + R'^{sri, S} = 1}} \sigma_S - \sum_{\substack{S \in \mathcal{S}: k \in S, i^tr \in S \\ R^{sri, S} = R'^{sri, S} = 0}} \sigma_S \\ - \sum_{\substack{C \in \mathcal{C}: k \in C, i^tr \notin S \\ R^{cc, C} = R'^{cc, C} = 0}} \tau_C + (*)$$

in case of the cost-based objective and

$$\begin{aligned} \tilde{c}_r^{dur} = & R^{rdc,dur} + R'^{rdc,dur} + \max\{R^{dur} + R'^{dur}, t_{idr,k}^{dr} + t_{k,jdr}^{dr}\} \\ & + \pi_{itr} - \pi_k - \\ & \sum_{\substack{S \in \mathcal{S}: k \in S, i^{tr} \notin S \\ R^{stri,S} + R'^{stri,S} = 1}} \sigma_S - \sum_{\substack{S \in \mathcal{S}: k \in S, i^{tr} \in S \\ R^{stri,S} = R'^{stri,S} = 0}} \sigma_S - \sum_{\substack{C \in \mathcal{C}: k \in C, i^{tr} \notin S \\ R^{cc,C} = R'^{cc,C} = 0}} \tau_C + (*) \end{aligned}$$

in case of the duration objective. Note that CCs have a nonpositive dual price τ_C so that they can lower the reduced cost when added in (*). Therefore, the acceleration of the merge procedure by stopping the loop over $k \in N(R, R', i^{tr})$ prematurely (see Section 4.2.6) must be altered; only if the reduced cost plus the sum $\sum_C \tau_C$ is nonnegative, the loop can be stopped prematurely. This is a rather weak condition.

Violated SRIs can be identified with a straightforward approach that enumerates all subsets S with $|S| = 3$ and then determines the left-hand side of (11), checks whether it is greater than one, and stores S together with the degree of violation. Separating violated CCs requires more computational effort. For the capacitated arc routing problem, Martinelli, Poggi, and Subramanian (2013) introduced an MIP-based CC separation algorithm that simultaneously computes C and K_C (for the above lower bound). The algorithm is directly applicable here.

Pretests have shown that adding many SRIs makes the labeling algorithm considerably more difficult. In contrast, the labeling algorithm can cope much better with CCs. Therefore, we always start with solving the separation problem for CCs and search for violated SRIs only if no CCs have been added. Moreover, we limit the total number of violated SRIs to be added to a maximum of 10 inequalities in total. Among the most violated inequalities, we allow no more than five per round and limit the number of SRIs per customer to not more than three. Violated inequalities are added only at the root node and we allow up to 100 CCs in total.

4.4. Branching

Let (λ_r^*, f^*) be a solution of the RMP. Whenever some component of (λ_r^*, f^*) is fractional, branching is necessary to obtain an integer solution. We apply the following four-level hierarchical branching scheme: (1) total number of routes, (2) number of drone visits at each customer, (3) flow of trucks on customer-to-customer arcs, and (4) flow on drone subpaths. At each branch-and-bound node, all routes that are incompatible with the current branching decision are temporarily removed from the RMP. The branch-and-bound tree is explored using a best-first search. Note that all constraints added to the RMP enforce additional dual prices in the labeling algorithm that all can be added on the corresponding arcs. We describe the four levels in detail now.

At the first level, when f^* is fractional, we branch on the total number of routes by creating two branches defined by $f \leq \lfloor f^* \rfloor$ and $f \geq \lceil f^* \rceil$, which is implemented by changing the bounds in (2d).

At the second level, we branch on the number of drone services performed for each customer (as proposed by Roberti and Ruthmair (2021) as a first-level branching decision in the algorithms for the TSP-D). To this end, let y_{kr} be the number of drone services performed by route r at customer k , which is the number of drone arcs of type $[\cdot, \cdot, k]$ used in the artificial network. If $y_k^* = \sum_{r \in \Omega} y_{kr} \lambda_r^*$ is fractional for some customers $k \in N$, we choose a customer k^* with value $y_{k^*}^* - \lfloor y_{k^*}^* \rfloor$ closest to 0.5 and branch on $y_{k^*} = 0$ and $y_{k^*} = 1$. Instead of adding a constraint to the RMP, these decisions can be enforced directly on the artificial network: the zero-branch results from eliminating all drone arcs of type $[\cdot, \cdot, k]$ serving customer k^* . Moreover, k^* must be removed from the sets $N(R, R', i^{tr})$ that are used in the merge procedure. The one-branch results from eliminating all vertices representing that customer k^* is served by a truck, that is, all vertices $(i^{tr}, i^{dr}) \in W$ with $i^{tr} = k^*$ or $i^{dr} = k^*$.

To finally ensure integrality, we use a similar branching strategy as proposed in Roberti and Ruthmair (2021); that is, we branch on the total flow induced by truck movements between the two customers and the total flow on drone subpaths, respectively. However, instead of branching on the directed flow (as proposed in Roberti and Ruthmair 2021), we branch on the total flow value.

At the third level, we consider pairs of customers $v, w \in N, v \neq w$ and the total flow induced by truck movements between the two customers. Formally, we consider the following subset of together arcs and alone arcs $A_{vw}^{tr} = \{[(v, v), (w, w), \perp], [(w, w), (v, v), \perp]\} \cup \{[i, j, \perp], [j, i, \perp] \in A^{alone} : i^{tr} = v, j^{tr} = w, \text{ and } i^{dr} = j^{dr} \in V\}$ and determine the values $e_{vw}^* = \sum_{r \in \Omega} \sum_{a \in A_{vw}^{tr}} e_{ar} \lambda_r^*$ for each customer pair of v and w , where e_{ar} is the number of times that route r uses arc $a \in A$. Note that the symmetry in the definition of A_{vw}^{tr} implies $e_{vw}^* = e_{vw}^*$ for all $v, w \in N, v \neq w$. If at least one of these values is fractional, we choose a customer pair v^* and w^* with value $e_{v^*, w^*}^* - \lfloor e_{v^*, w^*}^* \rfloor$ closest to 0.5 and create the two branches $\sum_{r \in \Omega} \sum_{a \in A_{v^*, w^*}^{tr}} e_{ar} \lambda_r = 0$ and $\sum_{r \in \Omega} \sum_{a \in A_{v^*, w^*}^{tr}} e_{ar} \lambda_r = 1$. As in the second level, the zero-branch can be directly enforced by removing all arcs A_{v^*, w^*}^{tr} from the artificial network. The one-branch needs to be enforced by adding the above constraint to the RMP in the same spirit as the one-branch needs to be enforced by an additional constraints when branching on undirected edges in other VRP variants.

In addition, it is feasible and advantageous to reduce the artificial network by removing further arcs that are incompatible with the branching constraint. In particular, the enforced truck customers v^* and w^* must never be served by a drone so that the arcs of type $[\cdot, \cdot, v^*] \in$

A^{drone} and $[\cdot, \cdot, w^*] \in A^{drone}$ can be eliminated. Likewise, v^* and w^* are disregarded as drone customers in the merge procedure. Branching decisions on the third level finally ensure unique truck paths. However, drone subpaths can still be fractional.

Hence, at the fourth level, we consider the total flow on drone subpaths for branching. Recall that any drone subpath $\langle v, k, w \rangle$ has a symmetric counterpart $\langle w, k, v \rangle$, which is uniquely determined by the two drone arcs $[(w, v), (w, w), k]$ and $[(v, w), (v, v), k] \in A^{drone}$. We define $A_{\langle v, k, w \rangle}^{dr} = \{[(w, v), (w, w), k], [(v, w), (v, v), k]\}$ and determine the values $e_{\langle v, k, w \rangle}^* = \sum_{r \in \Omega} \sum_{a \in A_{\langle v, k, w \rangle}^{dr}} e_{ar} \lambda_r^*$ for each drone subpath $\langle v, k, w \rangle$ with $v \neq k \neq w, v \neq w$. Note that the symmetry in the definition of $A_{\langle v, k, w \rangle}^{dr}$ implies $e_{\langle v, k, w \rangle}^* = e_{\langle w, k, v \rangle}^*$. We apply a similar strategy as at the third level: for a subpath $\langle v^*, k^*, w^* \rangle$ with value $e_{\langle v^*, k^*, w^* \rangle}^{dr} - [e_{\langle v^*, k^*, w^* \rangle}^{dr}]$ closest to 0.5, two branches defined by $\sum_{r \in \Omega} \sum_{a \in A_{\langle v^*, k^*, w^* \rangle}^{dr}} e_{ar} \lambda_r = 0$ and $\sum_{r \in \Omega} \sum_{a \in A_{\langle v^*, k^*, w^* \rangle}^{dr}} e_{ar} \lambda_r = 1$, respectively, are created. Again, the zero-branch results from removing the arcs $A_{\langle v^*, k^*, w^* \rangle}^{dr}$ from the artificial network. The one-branch is enforced by adding the corresponding constraint to the RMP. As before, the artificial network can be reduced by eliminating arcs that are inconsistent with this branching decision: the vertices $(i^{tr}, i^{dr}) \in W$ with $i^{tr} = k^*$ or $i^{dr} = k^*$ can be removed because the customer k^* is not served by truck. Further, all drone arcs of type $[\cdot, \cdot, v^*]$ and $[\cdot, \cdot, w^*]$ can be eliminated because the drone never serves the truck customers v^* and w^* . Finally, v^* and w^* must be disregarded as drone customers in the merge procedure.

The branching decisions at the third and fourth levels imply unique truck paths and drone subpaths. Because a solution is fully determined by both types of decisions, this four-level branching scheme is complete.

5. Computational Results

The following experiments were conducted on HPC cluster MOGON 2 of Johannes Gutenberg University Mainz using nodes equipped with Intel Xeon E5-2630v4 (Broadwell) processors running at 2.2 GHz. Notice that the performance of a single thread of the cluster is slightly worse compared with that of a standard desktop processor. The BPC algorithm is coded in C++ and compiled with GCC 11.2.0 in release mode into a 64-bit single-thread executable. The callable library of CPLEX 20.1.0 is used for the (re)optimization of the RMPs for solving a primal MIP-based heuristic and for separating CCs. CPLEX's default parameters are kept except for setting the number of threads to one. The MIP-based heuristic solves a restricted-integer MP using all columns generated up to this point. It is applied at the first and second levels of the branch-and-bound-tree as well as if the BPC algorithm terminates without finding a feasible solution. The BPC computation time is limited to

3,600 seconds per instance, and an additional 60 seconds are granted for the MIP-based heuristic at the end.

In what follows, Section 5.1 describes the VRP-D instances, Section 5.2 analyzes the impact of the acceleration techniques for the merge, Section 5.3 compares the pure forward with the new implicit bidirectional labeling algorithm, and Section 5.4 analyzes what effects result from different routing costs and speeds of the drones. Pointers to further results that are presented in the e-companion are given in Section 5.5.

5.1. VRP-D Instances

Until date, no suitable VRP-D benchmark set has been made available. Therefore, we created new VRP-D instances based on the *Capacitated Vehicle Routing Problem Library* (CVRPLIB)¹ to properly evaluate the performance of our BPC algorithm. We use CVRPLIB instances from *set A* and *set B* (proposed by Augerat et al. 1995) and consider the first 20, 30, 40, or 50 vertices, respectively. The first vertex always represents the depot. As a result, instances have $m = 19, 29, 39$, or 49 customers. For each size m , we construct 20 instances so that the VRP-D instance set comprises 80 instances.

VRP-D-specific data are determined in the following way. All customers with a demand greater than $\bar{q} = Q/5$ are truck-only customers. The share of truck-only customers varies between 10% and 33%, with an average of 20% over all instances. As a side note, if the value is kept in that interval, we could not see predictable differences in the difficulty of the instances. This statement can be proven with the instance-specific results presented in Tables EC.5–EC.10 in the e-companion. The literature stresses that routing costs and travel times are lower for drones than for trucks. Hence, we compute c_{ij} and t_{ij} for trucks as Manhattan distances. In contrast, c_{ij}^{dr} and t_{ij}^{dr} for drones are computed as Euclidean distances divided by the given drone speed factor β . We set $\beta = 3$ as the default value and analyze the impact of the drones' speed in Section 5.4 using the alternative factors $\beta = 1$ and $\beta = 5$.²

5.2. Acceleration of the Merge Procedure

When using implicit bidirectional labeling, the merge procedure is the bottleneck operation of the entire BPC algorithm. Therefore, we first examine the effect of the acceleration techniques presented in Section 4.2.6 on the time spent in the merge procedure. The following four algorithmic setups are considered: (*None*), no acceleration technique is used for the merge; (*Pre*), the reduced cost of each possible drone subpath is precomputed; (*Sort*), all labels belonging to the same vertex are sorted by their load resource; and (*Both*), both acceleration techniques are combined.

Figure 3 shows the percentage of the total BPC computation time spent on the merge operation depending on which of the above acceleration techniques is used.

The results are aggregated over instances of the same size. For all setups, the share of the merge time increases with the size m of the instances. Similar values are obtained for both objectives and setup *None*, where the merge accounts for 65%–85% of the total BPC runtime.

The results differ significantly by objective, with a relatively strong positive effect for cost minimization (Figure 3(a)) and a moderate effect for duration minimization (Figure 3(b)). The precomputation of the reduced cost of drone subpaths (*Pre*) helps to reduce the merge time by about seven percentage points on average only for cost minimization, whereas the reduction is negligible for duration minimization. The latter can be explained by the fact that prematurely stopping the loop over the drone customers $k \in N(R, R', i^{\text{tr}})$ for the merge at pairs of vertices where truck and drone are at different locations is not possible in case of duration minimization. The effect of sorting (*Sort*) the labels by their load resource is beneficial for both objectives (more so for duration than for cost minimization), as it reduces the share by about 8.7 and 9.6 percentage points, respectively, compared with setup *None*. However, even though the absolute savings are greater for *Sort* than for *Pre*, the respective savings in relative computation times decrease with the size m of the instances. The combination of both techniques (*Both*) reduces the merge times considerably: compared with setup *None*, they are halved in the best case of cost minimization and $m = 19$. In all the following experiments, the setup *Both* becomes the default.

5.3. Comparison of Forward and Bidirectional Labeling

We perform two types of computational experiments to compare the performance of the forward and the implicit bidirectional labeling strategies in the BPC algorithm. The first experiment focuses on quantifying their computational effort when applied to completely identical instances of the pricing problem. The second experiment then examines the impact on independent BPC runs.

In the first analysis, we consider the root node, that is, the linear relaxation of the MP, including the strengthening with the dynamic extension of the *ng*-neighborhoods, SRIs, and CCs. In each pricing iteration, the SPPRC subproblem is solved using both labeling strategies (forward and implicit bidirectional), applying the labeling algorithms consecutively. Because the dual prices and reduced costs used in both labeling algorithms are identical, a perfectly fair comparison is ensured. We add the negative reduced cost routes generated by the bidirectional algorithm to the RMP (this choice is arbitrary). As performance measures to compare the implicit bidirectional and forward labeling, we use the ratios of (1) the number of generated labels, (2) the number of processed labels, and (3) the total pricing time in the two labeling algorithms. The ratios are aggregated by taking the geometric mean over all pricing iterations of a VRP-D instance. Whenever the linear relaxation is not completely solved within the time limit, we omit the last iteration for both strategies. Table 1 reports the minimum (min), geometric mean (mean), and maximum (max) of each measure, aggregated over VRP-D instances with an identical number of customers. Additionally, the table lists the arithmetic mean (average) of the number of labeling algorithms called while solving the LP relaxation of the MP.

All *mean* and *avg* ratios presented in the table are less than one, which shows the implicit bidirectional labeling strategy is superior to its monodirectional counterpart. The number of calls is sufficiently large to provide statistically significant results. The number of generated (processed) labels reduces to 29% (23%) on average, that is, by more than a factor of three (four). These reductions do not fully translate to the time consumed by the labeling algorithms, which is reduced to 58% on average.

In the second analysis, we compare the two fully fledged BPC algorithms described in Section 4, where one uses the forward labeling and the other the implicit bidirectional labeling algorithm. Detailed results for each instance can be found in Tables EC.5–EC.10 in the e-companion. Table 2 aggregates these results obtained

Figure 3. (Color online) Average Share (in Percent) of the Total Computation Time Spent in the Merge Procedure for Different Acceleration Techniques

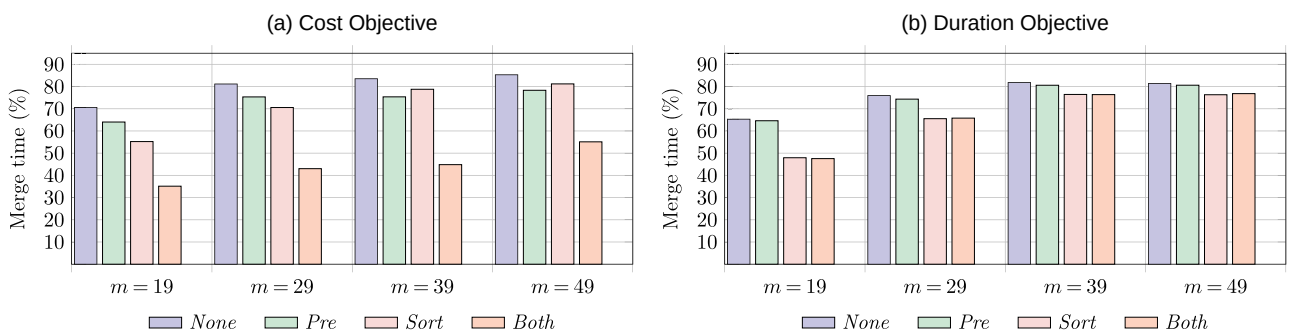


Table 1. Comparison of Forward and Implicit Bidirectional Labeling Using Identical Dual Prices

Objective	m	Ratio of:											
		Generated labels			Processed labels			Pricing time			Number of calls		
		Min	Mean	Max	Min	Mean	Max	Min	Mean	Max	Min	Average	Max
Cost	19	0.29	0.32	0.36	0.22	0.26	0.30	0.39	0.44	0.56	46	211.6	336
	29	0.27	0.29	0.33	0.21	0.22	0.29	0.36	0.43	0.58	149	390.1	513
	39	0.26	0.27	0.32	0.19	0.21	0.25	0.33	0.45	0.73	385	471.0	549
	49	0.24	0.26	0.30	0.18	0.21	0.25	0.32	0.46	0.60	428	517.3	579
Subtotal			0.29			0.22			0.45			397.5	
Duration	19	0.28	0.32	0.36	0.21	0.25	0.31	0.40	0.57	0.70	108	292.2	401
	29	0.25	0.29	0.33	0.19	0.22	0.27	0.43	0.67	1.16	246	376.2	471
	39	0.24	0.28	0.34	0.18	0.22	0.28	0.47	0.82	1.46	338	448.4	556
	49	0.21	0.26	0.32	0.16	0.20	0.26	0.34	0.75	1.20	203	371.9	495
Subtotal			0.29			0.23			0.70			372.2	
Total			0.29			0.23			0.58			384.9	

with both BPC algorithms, again grouped by VRP-D instances with an identical number m of customers. The column entries have the following meaning: ($\#Inst$), number of instances per group; ($\#Opt$), number of instances solved to proven optimality within the time limit; (Gap_{LP}), average LP gap ($100 \cdot (opt - z_{LP}) / z_{LP}$) in percent; (Gap_{cut}), average LP gap after extending the ng -neighborhoods and adding cutting planes in percent; (Gap), average relative difference between the upper bound (UB) and lower bound (LB) in percent at termination, that is, $100 \cdot (UB - LB) / LB$ (computed only when an upper and lower bound was found); ($Time$), average solution time in seconds; and ($\#BB$), average number of branch-and-bound nodes solved per VRP-D instance. Note that $Time$ includes unsolved instances with 3,600 seconds for the BPC and extra time for the MIP-based heuristic (see first paragraph of Section 5). For both Gap_{LP} and Gap_{cut} , we only consider instances with a known optimal solution opt .

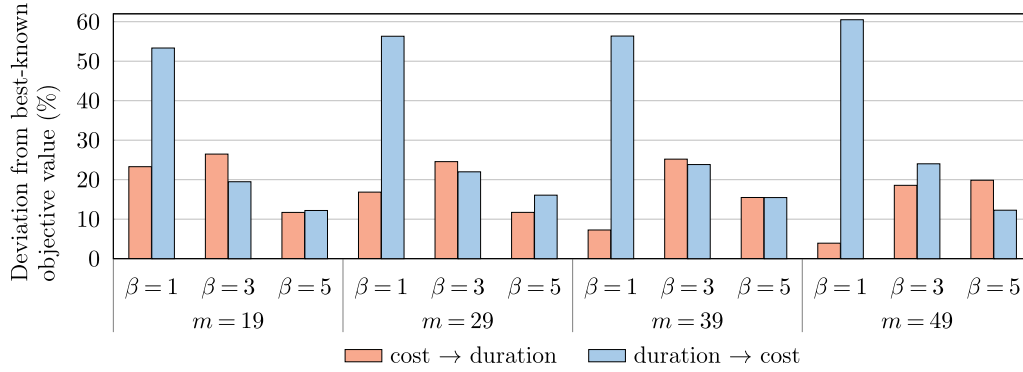
The results of the second analysis are very consistent over different instance sizes. For all but six instances with a duration objective, the BPC algorithm determines a feasible solution and herewith an upper bound UB . The implicit bidirectional labeling algorithm

outperforms the forward labeling algorithm; that is, more instances are solved to optimality (85 versus 75), average gaps and computation times are smaller, and more branch-and-bound nodes are processed (in a shorter time). In total, the BPC algorithm with implicit bidirectional labeling is on average around 15% faster than the BPC with forward labeling. The speedup for the smallest instances with $m = 19$ customers is more than by a factor of nine. The results for the instances with more customers are biased because of the one-hour time limit. Hence, the implicit bidirectional labeling is even more valuable than the 15% speedup suggests: for instances with $m = 29$ (39) customers, the number of proven optimal solutions for both objectives increases from 23 to 29 (11 to 14), and for the largest instances with $m = 49$ customers, approximately 75% more branch-and-bound nodes can be processed reducing the average gap from 10.08% to 1.63%. The results also confirm empirically that the same ng -route relaxation may have a stronger LP relaxation when bidirectional labeling (instead of forward) is applied (see Example 8); that is, Gap_{LP} is slightly smaller for the bidirectional labeling. To be more precise, for the cost (duration) objective, 52 (22) out of 55 (30) instances solved to optimality have a

Table 2. Comparison of Two BPC Algorithms Equipped with a Forward or Implicit Bidirectional Labeling Algorithm

Objective	m	Forward labeling						Implicit bidirectional labeling					
		$\#Opt$	Gap_{LP}	Gap_{cut}	Gap	$Time$	$\#BB$	$\#Opt$	Gap_{LP}	Gap_{cut}	Gap	$Time$	$\#BB$
Cost	19	20	3.19	0.27	0.00	206.4	42.9	20	2.92	0.30	0.00	22.3	37.7
	29	16	3.67	0.56	0.11	1,454.0	90.3	18	3.43	0.62	0.02	814.8	163.4
	39	11	2.76	0.22	0.84	2,638.1	84.0	14	2.59	0.26	0.16	1,634.5	141.1
	49	3	2.31	0.04	24.49	3,406.5	54.7	3	2.28	0.06	3.34	3,161.5	84.9
Subtotal		50	3.19	0.34	6.36	1,926.3	67.9	55	2.97	0.38	0.88	1,408.3	106.8
Duration	19	18	3.70	1.24	0.06	726.2	155.8	19	3.59	1.42	0.01	335.7	232.7
	29	7	2.14	0.96	3.52	2,900.3	142.4	11	2.11	1.00	1.27	2,486.0	325.6
	39	0	—	—	14.04	3,602.1	53.6	0	—	—	2.29	3,640.9	95.4
	49	0	—	—	37.56	3,601.7	13.8	0	—	—	5.92	3,655.2	33.3
Subtotal		25	3.13	1.14	12.54	2,707.6	91.4	30	3.04	1.26	2.13	2,529.5	171.7
Total		75			10.08	2,316.9	79.7	85			1.63	1,968.9	139.2

Figure 4. (Color online) Increase of the Objective Value when Changing the Objective (Cost vs. Duration Minimization)



stronger *LP relaxation*. All experiments conducted in the following use the implicit bidirectional labeling algorithm to solve the pricing subproblems.

5.4. Effect of the Drones' Routing Cost and Speed

The routing cost and speed of drones compared with trucks have a strong influence on the structure of optimal and near-optimal solutions. We examine the impact in four analyses.

5.4.1. Conflicting Objectives. First, we show that the two objectives are conflicting. To this end, we reevaluate optimal and best-known solutions for one objective using the other objective. These values are then compared with best-known solutions for the other objective. Figure 4 shows the resulting average percentage increase in objective value for different values of β and m .

Considering the evaluation of the solutions optimized with respect to routing cost using the duration objective (labeled with “cost → duration”), we find the highest average deviation of 23.7% for $\beta = 3$. Results for $m = 49$ should be treated with caution because the number of optimal solutions is small so that best-known upper bounds are used instead.

For the reverse evaluation (labeled with “duration → cost”), the cost increase for $\beta = 1$ stands out. Solutions optimized with respect to duration are (on average) rather bad solutions for routing-cost minimization. An explanation is related to the analysis presented in the next paragraph (see Table 3 and Example 9), where we highlight that about 43% of all customers are served by a drone when it comes to duration minimization. However, when the cost is minimized, drone flights are not profitable. Consequently, converting a duration solution into a corresponding cost solution will drastically increase the cost. For $\beta = 3$ and $\beta = 5$, the deviations resulting from the two reevaluations are in a rather similar range for all instance sizes.

5.4.2. Share of Drone Customers. We now analyze the structure of optimal and best-known solutions. As

shown in Table 3, the share of drone customers varies considerably depending on the objective, the size of the instance, and the β -value. On average, the share increases with β and with the substitution of cost by duration minimization. In turn, the share decreases slightly with increasing instance size. The influence of the parameter β is as expected because cheaper and faster drones encourage their use.

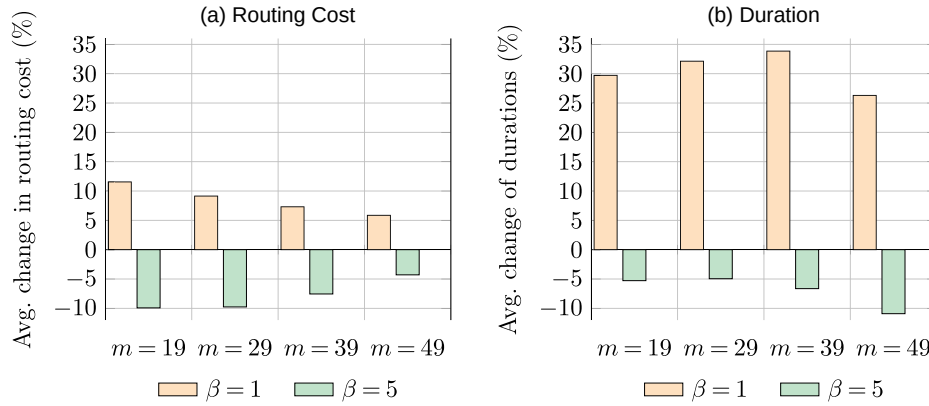
The most surprising and striking result is that drones are hardly ever used when the routing cost is minimized and trucks and drones have identical cost/speed characteristics ($\beta = 1$). The following example explains the above result.

Example 9. Consider a tiny VRP-D instance with three customers $N = \{1, 2, 3\}$. The complete undirected graph is then $G = (V, E)$ with $V = \{0, 1, 2, 3, 0'\}$. Recall from Section 5.1 that routing costs and travel times are computed from Manhattan distances for trucks and Euclidean distances for drones. Accordingly, we define $t_{ij} = c_{ij} = 2$ and $t_{ij}^{\text{dr}} = c_{ij}^{\text{dr}} = 1.5/\beta$ for all $\{i, j\} \in E$. (Similar distances of 2 and $1.5 \approx \sqrt{2}$ occur when the coordinates in x - and y -directions differ by one.)

When all three customers are served by the same truck, the corresponding route r has an objective value of $c_r = 8$, regardless of the order in which the customers are visited. This is true for both objectives. Alternatively, we assume that one of the customers is served by the drone and the other two by the same truck to which the drone is assigned. The corresponding route is denoted by r' . Its drone subpath has a routing cost of $2 \cdot 1.5/\beta$, and the duration increases by $\max\{2, 2 \cdot 1.5/\beta\}$. Hence,

Table 3. Share of Drone Customers in Optimal/Best-Known Solutions

M	Cost objective			Duration objective		
	$\beta = 1$	$\beta = 3$	$\beta = 5$	$\beta = 1$	$\beta = 3$	$\beta = 5$
19	0%	27%	41%	29%	51%	55%
29	0%	27%	40%	25%	52%	55%
39	0%	22%	38%	26%	49%	52%
49	1%	23%	32%	26%	46%	48%

Figure 5. (Color online) Average Percentage Change in Routing Cost and Duration for Different β -Values

the routing cost accumulates to $c_r = 6 + 3/\beta$, and the duration is $t_r = 4 + \max\{2, 3/\beta\}$.

For $\beta < 1.5$, it follows that $c_r < c_r'$ for routing costs so that using the drone is not beneficial (in particular for $\beta = 1$). This relationship is reversed for $\beta > 1.5$ so that the use of the drone is cost-effective (in particular, for $\beta \in \{3, 5\}$). This reasoning is empirically proven by the results shown on the left side of Table 3. For the duration objective, the first travel time, $t_r = 8$, is never smaller than the second travel time, $t_r' = 4 + \max\{2, 3/\beta\}$ (assuming that the drone speed is never lower than the truck speed, i.e., $\beta \geq 1$). Therefore, it is always advantageous to use the drone, as can be seen from the results on the right side of Table 3. $\square$

5.4.3. Comparison of Different β -Values. Next, we compare optimal and best-known solutions for $\beta = 3$ with optimal and best-known solutions for the values $\beta = 1$ and $\beta = 5$, respectively. Figure 5 visualizes the average percentage changes relative to the baseline solutions for $\beta = 3$. First and foremost, the percentage changes are smaller for cost minimization (Figure 5(a)) and larger for the duration minimization (Figure 5(b)). For routing costs, increasing the drones' routing costs by a factor of 3 (from $\beta = 3$ to $\beta = 1$) results in a total cost increase between 5% and 12%, depending on the instance size. Reducing the routing cost of the drones (from $\beta = 3$ to $\beta = 5$) generates cost savings of about the same amount. For duration minimization, the corresponding changes are between 25% and 35% and between 5% and 11%, respectively. The latter result implies that drones should be faster than trucks, but further increasing their speed beyond $\beta = 5$ will not generate substantial additional savings.

5.4.4. Comparison with Approximation Results. The above results are in line with those of Carlsson and Song (2018), namely, that the efficacy of the combined truck-and-drone delivery increases with the ratio of the speeds of the drones and trucks. Carlsson and Song

(2018)'s most important finding is that “the improvement in efficiency [resulting from the use of a drone] is proportional to the square root of the ratio of the speeds of the truck and the unmanned aerial vehicle” [drone]. Carlsson and Song (2018) do not use MIP-based optimization techniques but a continuous approximation model (see Daganzo 2005). They measure efficiency as the duration of a tour (a *max*-term as in the Duration Objective (1)). However, in their model, only the drone delivers to the customers, and the truck serves as a support vehicle (as in case 1 in Section 1). When reducing the drone speed from $\beta = 3$ to $\beta = 1$, Carlsson and Song (2018)'s model predicts a 73% increase in duration ($\sqrt{3} = 1.73$). Our results show a relatively smaller duration increase of 26% on average, which can be partly explained by the fact that, in our setting, only about half of the customers can be served by the drones (see Table 3). Increasing the drone speed from $\beta = 3$ to $\beta = 5$, they predict a 23% reduction in duration ($\sqrt{3/5} = 0.77$). Our results are then rather consistent with the above numbers showing an average duration reduction of only 6.2%.

5.5. Further Results

Roberti and Ruthmair (2021) model and solve a variety of constraints and options that are meaningful for the TSP-D. They mention (1) loops (the drone is allowed to return to the location where it was launched), (2) incompatible customers (customers who must be served by a truck), (3) the drone cannot land and wait for the truck, (4) launch and rendezvous times (additional time to be added when a drone is launched from or lands at a customer location), (5) a maximum number of customers per truck leg, and (6) a limited drone flight range. These extensions of the basic TSP-D are certainly also relevant for the multiple-truck case. Because our artificial network is based on the network introduced by Roberti and Ruthmair (2021), the same DP adaptations can be used here. All variants except for variant 5 can be handled by restricting the label propagation and/or

restricting the set of drone arcs, whereas variant 5 requires the introduction of an additional attribute into the labels. We have exemplarily tested our algorithm with a limited drone flight range. See Section EC.1 in the e-companion for details. Additionally, we have tested the performance of our BPC on the instance set proposed by Zhen et al. (2023). Note that Zhen et al. (2023) present one of the exact BPC algorithms that is applicable to the basic VRP-D and closely related variants. Results can be found in Section EC.2 in the e-companion. Finally, we considered the TSP-D instances of Roberti and Ruthmair (2021) and Blufstein, Lera-Romero, and Soullignac (2024) and solved them twice as VRP-D instances, once with a nonbinding capacity and once with capacity of $Q = 8$ and unit demand for all customers. Results are presented in Section EC.3 of the e-companion.

6. Conclusions

In this work, we presented a new exact solution algorithm for the basic VRP-D that is based on BPC. Its most important algorithmic component is a new implicit bidirectional labeling algorithm for solving pricing subproblems based on an artificial network originally introduced by Roberti and Ruthmair (2021) for TSP-D. In this artificial network, the same route, once considered in the forward and once in the backward direction, passes through different vertices. Our new implicit bidirectional labeling algorithm exploits the symmetry inherent in the VRP-D, as it, however, does not distinguish forward and backward directions. Roberti and Ruthmair (2021) have shown how to modify the artificial network to cope with TSP-D variants that consider, for example, drone loops in addition to drone subpaths, a (possibly weight-dependent) drone flight range, and a maximum number of customers per truck segment. Using the same ideas, our approach is adaptable to the corresponding VRP-D variants.

For the performance of the BPC algorithm, several known techniques had to be adapted to the VRP-D case: delayed propagation of some resource values is helpful to handle the nonrobust cutting planes (subset-row inequalities and capacity cuts) that strengthen the linear relaxation of the column-generation MP. In addition, the performance of the new bidirectional labeling algorithm is improved by problem-tailored preprocessing and acceleration techniques of the merge procedure, which is otherwise the bottleneck of the overall solution algorithm. At the end, the BPC algorithm is able to exactly minimize the routing cost for 14 (3) out of the 20 instances of the VRP-D with 39(49) customers. The minimization of the duration is more difficult such that only half of the instances with up to 29 customers can be solved to optimality. However, feasible solutions with an average gap of around 2.3% (5.9%) are found for instances with 39 (49) customers. Compared with other exact algorithms for variants of the VRP-D (see Section 2)

that allow solution times of up to 3, 5, 10, and even 20 hours for instances with not more than 35 customers, our new BPC algorithm obtains the reported results within one hour.

Our results should also be of interest to other emerging variants of combined vehicle routing. For example, the VRP-D considered here shares many similarities with routing problems in last-mile delivery with trucks and delivery robots (see, e.g., Boysen, Schwerdfeger, and Weidinger 2018). For the future, the introduction of time windows and the consideration of time-related objective functions (such as total (weighted) tardiness or lateness or the makespan) into VRP-D variants makes the truck-drone synchronization aspect even more challenging, especially when considering the exact solution of these problems.

Acknowledgments

The authors gratefully acknowledge the computing time granted on the supercomputer MOGON 2 at Johannes Gutenberg University Mainz (hpc.uni-mainz.de).

Endnotes

¹ The CVRPLIB is publicly available at <http://vrp.atd-lab.inf.puc-rio.br/index.php/en/>.

² The new VRP-D instances with details on how costs and times are computed can be found at <https://logistik.bwl.uni-mainz.de/research/benchmarks/>.

References

- Augerat P, Belenguer JM, Benavent E, Corberán A, Naddef D, Rinaldi G (1995) Computational results with a branch and cut code for the capacitated vehicle routing problem. Technical report, Institut National Polytechnique, Grenoble, France.
- Bakır İ, Tiniç GÖ (2020) Optimizing drone-assisted last-mile deliveries: The vehicle routing problem with flexible drones. *Optim. Online* (April 9), <https://optimization-online.org/?p=16382>.
- Baldacci R, Christofides N, Mingozzi A (2008) An exact algorithm for the vehicle routing problem based on the set partitioning formulation with additional cuts. *Math. Programming* 115(2):351–385.
- Baldacci R, Mingozzi A, Roberti R (2011) New route relaxation and pricing strategies for the vehicle routing problem. *Oper. Res.* 59(5):1269–1283.
- Baldacci R, Mingozzi A, Roberti R (2012) New state-space relaxations for solving the traveling salesman problem with time windows. *INFORMS J. Comput.* 24(3):356–371.
- Blufstein M, Lera-Romero G, Soullignac FJ (2024) Incremental state-space relaxations for the basic traveling salesman problem with a drone. *INFORMS J. Comput.* 36(4):1064–1083.
- Bode C, Irnich S (2012) Cut-first branch-and-price-second for the capacitated arc-routing problem. *Oper. Res.* 60(5):1167–1182.
- Bode C, Irnich S (2015) In-depth analysis of pricing problem relaxations for the capacitated arc-routing problem. *Transportation Sci.* 49(2):369–383.
- Boysen N, Schwerdfeger S, Weidinger F (2018) Scheduling last-mile deliveries with truck-based autonomous robots. *Eur. J. Oper. Res.* 271(3):1085–1099.
- Carlsson JG, Song S (2018) Coordinated logistics with a truck and a drone. *Management Sci.* 64(9):4052–4069.
- Chung SH, Sah B, Lee J (2020) Optimization for drone and drone-truck combined operations: A review of the state of the art and future directions. *Comput. Oper. Res.* 123:105004.

- Costa L, Contardo C, Desaulniers G (2019) Exact branch-price-and-cut algorithms for vehicle routing. *Transportation Sci.* 53:946–985.
- Daganzo C (2005) *Logistics Systems Analysis*, 4th ed. (Springer, Berlin, Heidelberg).
- D'Andrea R (2014) Guest editorial: Can drones deliver? *IEEE Trans. Automation Sci. Engrg.* 11(3):647–648.
- Desaulniers G, Desrosiers J, Solomon M, eds. (2005) *Column Generation* (Springer, New York).
- Drexel M (2012) Synchronization in vehicle routing—A survey of VRPs with multiple synchronization constraints. *Transportation Sci.* 46(3):297–316.
- Gamache M, Soumis F, Marquis G, Desrosiers J (1999) A column generation approach for large-scale aircrew rostering problems. *Oper. Res.* 47(2):247–263.
- Goeke D, Gschwind T, Schneider M (2019) Upper and lower bounds for the vehicle-routing problem with private fleet and common carrier. *Discrete Appl. Math.* 264:43–61.
- Goodchild A, Toy J (2018) Delivery by drone: An evaluation of unmanned aerial vehicle technology in reducing CO₂ emissions in the delivery service industry. *Transportation Res. Part D Transport Environ.* 61:58–67.
- Heßler K, Irnich S (2023) Partial dominance in branch-price-and-cut for the basic multicompartment vehicle-routing problem. *INFORMS J. Comput.* 35(1):50–65.
- International Trade Administration (2023) Unmanned aircraft systems (UAS). Accessed May 3, 2023, <https://www.trade.gov/unmanned-aircraft-systems>.
- Irnich S (2008) Resource extension functions: Properties, inversion, and generalization to segments. *OR Spectrum* 30(1):113–148.
- Irnich S, Desaulniers G (2005) Shortest path problems with resource constraints. Desaulniers G, Desrosiers J, Solomon MM, eds. *Column Generation* (Springer, New York), 33–65.
- Irnich S, Toth P, Vigo D (2014) The family of vehicle routing problems. Toth P, Vigo D, eds. *Vehicle Routing: Problems, Methods, and Applications* (Society for Industrial and Applied Mathematics, Philadelphia), 1–33.
- Jepsen M, Petersen B, Spoorendonk S, Pisinger D (2008) Subset-row inequalities applied to the vehicle-routing problem with time windows. *Oper. Res.* 56(2):497–511.
- Joerss M, Schröder J, Neuhaus F, Klink C, Mann F (2016) Parcel delivery: The future of last mile. McKinsey & Company. https://bdkep.de/files/bdkep-dateien/pdf/2016_the_future_of_last_mile.pdf.
- Lera-Romero G, Miranda Bront JJ, Soullignac FJ (2022) Dynamic programming for the time-dependent traveling salesman problem with time windows. *INFORMS J. Comput.* 34(6):3292–3308.
- Li H, Wang F (2022) Branch-price-and-cut for the truck-drone routing problem with time windows. *Naval Res. Logist.* 70(2):184–204.
- Macrina G, Di Puglia Pugliese L, Guerriero F, Laporte G (2020) Drone-aided routing: A literature review. *Transp. Res. Part C Emerging Tech.* 120:102762.
- Madani B, Ndiaye M (2022) Hybrid truck-drone delivery systems: A systematic literature review. *IEEE Access* 10:92854–92878.
- Martinelli R, Poggi M, Subramanian A (2013) Improved bounds for large scale capacitated arc routing problem. *Comput. Oper. Res.* 40(8):2145–2160.
- Moshref-Javadi M, Winkenbach M (2021) Applications and research avenues for drone-based models in logistics: A classification and review. *Expert Systems Appl.* 177:114854.
- Otto A, Agatz N, Campbell J, Golden B, Pesch E (2018) Optimization approaches for civil applications of unmanned aerial vehicles (UAVs) or aerial drones: A survey. *Networks* 72(4):411–458.
- Poikonen S, Wang X, Golden B (2017) The vehicle routing problem with drones: Extended models and connections. *Networks* 70(1):34–43.
- Righini G, Salani M (2006) Symmetry helps: Bounded bi-directional dynamic programming for the elementary shortest path problem with resource constraints. *Discrete Optim.* 3(3):255–273.
- Roberti R, Mingozzi A (2014) Dynamic ng-path relaxation for the delivery man problem. *Transportation Sci.* 48(3):413–424.
- Roberti R, Ruthmair M (2021) Exact methods for the traveling salesman problem with drone. *Transportation Sci.* 55(2):315–335.
- Tamke F, Buscher U (2021) A branch-and-cut algorithm for the vehicle routing problem with drones. *Transportation Res. Part B Methodological* 144:174–203.
- Tilk C, Irnich S (2017) Dynamic programming for the minimum tour duration problem. *Transportation Sci.* 51(2):549–565.
- Tilk C, Rothenbächer AK, Gschwind T, Irnich S (2017) Asymmetry matters: Dynamic half-way points in bidirectional labeling for solving shortest path problems with resource constraints faster. *Eur. J. Oper. Res.* 261(2):530–539.
- Valério de Carvalho JM (1999) Exact solution of bin-packing problems using column generation and branch-and-bound. *Ann. Oper. Res.* 86:629–659.
- Wang Z, Sheu JB (2019) Vehicle routing problem with drones. *Transportation Res. Part B Methodological* 122:350–364.
- Wang X, Poikonen S, Golden B (2017) The vehicle routing problem with drones: Several worst-case results. *Optim. Lett.* 11(4):679–697.
- Xia Y, Zeng W, Zhang C, Yang H (2023) A branch-and-price-and-cut algorithm for the vehicle routing problem with load-dependent drones. *Transportation Res. Part B Methodological* 171:80–110.
- Zhen L, Gao J, Tan Z, Wang S, Baldacci R (2023) Branch-price-and-cut for trucks and drones cooperative delivery. *IIEE Trans.* 55(3):271–287.
- Zhou H, Qin H, Cheng C, Rousseau LM (2023) An exact algorithm for the two-echelon vehicle routing problem with drones. *Transportation Res. Part B Methodological* 168:124–150.

Copyright 2025, by INFORMS, all rights reserved. Copyright of Transportation Science is the property of INFORMS: Institute for Operations Research and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.